

TikTok — System Design 速成 (entry-level)

用途：TikTok entry-level system design 面速成。全中文、口语化（像面试对着人讲），专有名词保留英文（Redis / MongoDB / Kafka / CDN 等），代码/公式后紧跟一个括号把它读出来。结构：Part 1 答题框架（拿到任何问题都这么走）/ Part 2 积木速查（常用组件）/ Part 3 高频题库（10 道，每题：需求→估算→API→数据模型→高层架构→取舍瓶颈）。

Agent Notes（内部规则，勿当面试内容 / 给 agent 读的）

维护本文件遵守：

1. **全中文 + 口语化**：答题内容像面试现场一步步讲出来，第一人称（“我会先...”、“这里我用...”），不是书面技术文档。
2. **专有名词保留英文**：Redis / MongoDB / MySQL / load balancer / cache / Kafka / CDN / sharding / consistent hashing / QPS / API / websocket / bloom filter 等不翻译。
3. **代码/表达式/公式后加括号读出来**：如 $QPS = \text{日请求} / 86400$ （每天请求数除以一天的秒数）。
4. **深度=entry-level**：讲清组件、数据流、基本取舍即可，不深挖分片算法内部/一致性协议证明/CAP 数学。
5. **经历只做背书，不抢主线**：系统设计以常规设计为主，别把公司实践当独立论点展开。真实经历放在“被迫问时一句话带过”的位置，起可信度背书作用即可。Notification / Rate Limiter / 视频存储这三题挂得上（订阅 webhook、UGC 反作弊、视频上传全链路），Twitter 关注流那题挂不上——Newsbreak 是内容分发平台（爬取新闻为主、follow 是弱功能），不是社交网络，没有 fanout 那套，别硬靠。
6. 只在用户明确要时增删题目。

Part 1 — 答题框架（拿到任何问题都这么走）

面 system design 最怕的就是一上来直接画框、往里塞 Redis、塞 Kafka。面试官其实不太看你堆了多少组件，他看的是你有没有一套**稳定的思路**，能不能把一道模糊的开放题一步步收敛成一个能落地的设计。所以我强烈建议：不管拿到什么题，都用同一套流程走下来。下面这七步，就是我每次都会照着走的顺序。你把它练熟了，进场哪怕紧张，嘴上也有话说、手上也知道先画哪一块。

我先把节奏说一下：一场 45 分钟到 1 小时的面试，前面澄清 + 估算 + API + 数据模型这几步加起来大概花 10 到 15 分钟，中间画高层架构 5 到 10 分钟，剩下大头全砸在“深挖一两个组件”上。**千万别把时间平均分**，深挖那一段才是真正拉开分数的地方，前面几步是给深挖打地基的，不要在前面磨太久。

第 1 步：澄清需求，先别急着答

拿到题的第一件事不是设计，是问问题。题目一定是模糊的，比如“设计一个 Twitter”——那到底要不要发图、发视频？要不要私信？要不要 timeline 排序还是按时间倒序就行？这些不问清楚，你后面全在瞎设计。我会开口说：“我想先确认一下范围，我们这版是不是先聚焦在发帖和刷 timeline 这两个核心功能，私信、通知这些先放一放？”让面试官帮你圈定边界。

这一步面试官在看什么？他看你会不会**主动缩小范围**。真实工作里需求永远是模糊的，一个好工程师不会闷头就干，会先对齐。初学者最常犯的错就是**跳过这一步直接开画**，或者反过来——问了一堆无关紧要的细节（“用户名最多多少个字符”这种），显得抓不住重点。你要问的是**影响架构的大方向问题**，不是产品细节。

第 2 步：分清功能性需求和非功能性需求

澄清完之后，我会把需求明确地分成两类写在白板上。一类是**功能性需求**（functional requirements，就是系统“能干啥”），比如用户能发帖、能看别人的 timeline、能点赞。另一类是**非功能性需求**（non-functional

requirements，就是系统"要多好"），比如要支持多少用户、延迟要多低、能不能容忍短暂的数据不一致、可用性要几个九。

为什么要单独拎出来？因为**非功能性需求直接决定你后面所有的技术选型**。如果面试官说"读远远多于写、而且能接受刷到的 timeline 晚几秒更新"，那我立刻就知道后面可以大胆上 cache、可以做读写分离、可以接受最终一致性——这是一个巨大的信号。这一步面试官看的就是**你能不能把业务描述翻译成技术约束**。初学者常犯的错是只顾着列功能，完全不谈规模、延迟、一致性这些，结果后面架构选型全凭感觉，说不出"我为什么这么选"。

第 3 步：粗估规模——QPS 和存储

这一步是要拍几个数量级出来。注意关键词是"粗估"，面试官**不要求你算准，要的是你敢估、估的过程合理**。我一般从用户量倒推。比如假设日活 1 亿，每人每天平均发 2 条帖、刷 50 次 timeline。

写的 QPS 我会这么算：写 QPS = 每天写请求数 / 86400（也就是每天一共写多少次，除以一天有 86400 秒）。1 亿用户乘以每天 2 条，就是 2 亿次写，除以 86400 大概两千多，所以写 QPS 差不多 2000 到 3000 这个量级。读的 QPS 同理，读远大于写，可能到几十万。这时我会顺口说一句："读写比大概是一百比一，这说明这是个典型的读密集型系统，后面我会重点在读路径上做 cache。"——你看，估算不是为了算而算，是为了**指导后面的设计**。

存储也估一下：存储量 = 每天新增数据条数 × 每条大小 × 保存天数（就是每天产生多少数据，乘上一条多大，再乘以你要存多少天）。一条帖子假设 300 字节，一天 2 亿条，那一天就是几十 GB，一年下来就是几十 TB，这个量级立刻告诉你单机 MySQL 存不下，得考虑 sharding。这一步初学者最常见的毛病有两个：一是**完全跳过估算**直接设计，那你后面凭什么说要分库？二是在**算术上纠结太久**，把 86400 精确到个位——面试官根本不在乎，你可以直接说"我把它近似成 10 万秒好算"，这反而是加分的，说明你懂抓大放小。

第 4 步：定义 API

有了功能和规模，我会先把对外的 API 接口列出来，这是客户端和后端之间的契约。比如发帖就是一个 POST /tweet，请求体里带 user_id 和 content（也就是发一条帖子，带上是谁发的和发的内容）；刷 timeline 就是 GET /timeline?user_id=...&cursor=...（拉某个用户的 timeline，带一个翻页游标）。

为什么要先定 API？因为**API 一定下来，输入输出就清楚了，后面的数据模型和架构就有了锚点**。这里我会特意提一句翻页：timeline 这种长列表，我会用 cursor（游标）翻页而不是 offset（偏移量），因为数据一直在变，用 offset 翻页会翻重复或者漏掉。这种小细节面试官很喜欢，说明你有实战意识。初学者常犯的错是**API 设计得太随意**，或者干脆不定 API 直接跳到画框——那你后面组件之间传什么数据都说不清楚。

第 5 步：设计数据模型

接下来定数据怎么存、存在哪。我会先列核心的几张表或几个集合：用户表、帖子表、关注关系表。每张表列出关键字段和主键。然后**结合第 3 步的规模来选存储**：结构化、要事务的数据我放 MySQL；如果规模大到要分片，我会说清楚**按什么字段做 sharding**——比如帖子表按 user_id 做 sharding key（也就是用发帖人的 id 来决定这条数据落到哪个分片），这样同一个人的帖子都在一起，查起来快。读特别热的数据，比如某个大 V 的 timeline，我会在前面加一层 Redis 做 cache。

这一步面试官看的是你**存储选型有没有理由**，以及你知不知道 SQL 和 NoSQL 的基本取舍——需要强一致和事务用关系型，需要海量写入和灵活 schema、能接受最终一致就用 NoSQL。作为 entry-level，你不需要深入讲分片算法内部怎么用 consistent hashing 做的，但你得知道"数据大了要分片、分片要挑一个 key、挑不好会有热点"这个层次就够了。初学者常犯的错是**一上来就"用 NoSQL 因为它快"，问为什么快就答不上来了——选型一定要能说出取舍，不能是口号。

第 6 步：画高层架构图

现在才开始画框。从客户端出发，一路画到底层存储：客户端 → load balancer（负载均衡，把请求分发到多台机器）→ 一组无状态的应用服务器 → 后面接数据库、cache、还有异步处理用的消息队列（比如 Kafka）。数据流我会

边画边讲，比如："用户发帖，请求先到 load balancer，分到某台应用服务器，服务器把帖子写进 MySQL，同时往 Kafka 丢一条消息，让下游异步去更新粉丝的 timeline。"

这里有个关键点我一定会强调：**应用服务器要设计成无状态的**（stateless，就是服务器自己不存用户会话数据），这样任意一台都能处理任意请求，才能水平扩展、随便加机器。这一步面试官看的是你**脑子里有没有完整的数据流**，不能把前面几步的东西串成一个能跑通的整体。初学者常犯的错是**画得太满**——一上来就把限流、监控、CDN、服务发现全塞进去，图乱得自己都讲不清。高层架构要**干净**，先把主干画对，细节留到下一步深挖时再补。

第 7 步：深挖一两个组件 + 找瓶颈和扩展

这是重头戏，分数主要在这拿。前面架构画完，我会主动说："这个设计里我觉得最有意思的是 timeline 的生成，我想深入讲一下。" 然后挑一两个核心难点往深了聊。比如 timeline 到底是**读的时候现拉**（读扩散，pull）还是**发帖时就推给所有粉丝**（写扩散，fan-out on write）？我会讲这两种的取舍：写扩散读的时候快但大 V 发一条要推给几千万粉丝、写放大严重；读扩散写得轻但读的时候要现算、慢。然后给一个折中方案——普通用户用写扩散，大 V 用读扩散混合着来。**能讲出取舍、能给折中，这就是高分回答。**

深挖之后再回过头找瓶颈：哪块先扛不住？通常就是数据库读、或者某个热点数据。然后对应地给扩展手段——读扛不住就加 cache、加 read replica（只读副本）；写扛不住就 sharding；单点故障就做多副本 + 主从切换。这一步面试官看的是你**有没有能力自己发现系统的软肋并给出应对**，这是资深和初级的分水岭。初学者最常见的问题是**深挖不下去**——每个组件都只讲一句就滑走了，全程浮在表面，或者被面试官追问"这里为什么慢"就卡住。破解办法就是提前准备好几个经典难点（timeline 生成、热点 key、一致性、限流），面试时主动往这些你熟的地方引，把主动权抓在自己手里。

最后给一句总的心法：这七步的本质是**从模糊到具体、每一步的决定都要有前一步作为依据**。估算指导选型、选型指导架构、架构暴露瓶颈、瓶颈引出扩展——它是一条因果链，不是七个孤立的动作。你答题的时候如果能不断把这条链说出来（"因为读写比是一百比一，所以我在读路径加 cache""因为一年几十 TB，所以帖子表要 sharding"），面试官就会觉得你不是在背模板，而是真的在设计系统。这比你记住多少个组件名字都值钱。

Part 2 — 积木速查（面试就是搭这些积木）

其实 system design 面试到最后你会发现，来来回回就是那么十几个组件在不同题目里换着搭。你把每个积木的"是什么、什么时候拿出来用、有什么坑"想清楚，画图的时候就不慌了。我下面就一个个过一遍，讲的时候我会尽量像面试现场那样说。

Load balancer（负载均衡器）

load balancer 我一般会摆在客户端和一堆 server 之间，它的活儿就是把进来的请求分摊到后面多台机器上，别让某一台被打爆。什么时候用它？基本上只要我的服务是多台机器组成的、想做水平扩展，前面就得放一个 load balancer，不然客户端根本不知道该连哪台。它顺带还能做健康检查，某台机器挂了就自动把流量绕开。

最基础的一个取舍我会提一下轮询和一致性两种分发策略：如果我简单的 round-robin（轮流分，第一个请求给 A、第二个给 B、第三个给 C 这样转圈），实现最简单，但问题是如果 server 上存了用户的 session（会话状态），同一个用户下次请求可能被分到另一台就找不到自己的登录状态了。所以这里的坑就是：要么让 server 变成 stateless（无状态，把 session 存到外面的 Redis 里，谁接都一样），要么让 load balancer 做 session sticky（黏住，同一个用户一直发给同一台）。面试里我会倾向前者，因为 stateless 更好扩展。

Cache（缓存，一般用 Redis）

cache 就是把热的、经常被读的数据放到一个访问特别快的地方（通常是内存里的 Redis），这样请求来了先问 cache，命中了就不用去打后面的数据库。什么时候用？读远多于写、而且同一批数据被反复读的场景，比如热门文章、用户资料，这种最适合加 cache。加了以后数据库压力能降一大截，延迟也低。

基本取舍是：cache 里的数据可能和数据库不一致（数据库改了但 cache 还是旧的），所以我得想清楚过期策略，一般给个 TTL（存活时间，比如 60 秒后自动失效）。另外这里有三个经典的坑面试很爱问，我一句话各说一下：cache 穿透是查一个数据库里根本不存在的 key，cache 每次都不命中、请求全打到数据库（应对办法是把“不存在”这个结果也缓存一下，或者前面加个 bloom filter 先挡一道）；cache 雪崩是大量 key 在同一时刻集体过期，一瞬间请求全涌向数据库（应对是给过期时间加个随机抖动，别让大家同时到点）；cache 击穿是某一个超热的 key 过期了，那一瞬间海量请求同时去数据库回源（应对是加锁，只让一个请求去查库、其他人等它回填）。这三个名字容易混，我记的口诀是：穿透查的是不存在的、雪崩是一大片同时挂、击穿是一个热点单点挂。

SQL (MySQL) 还是 NoSQL (MongoDB) 怎么选

这题面试几乎必问，我不会背定义，我会讲什么时候选哪个。MySQL 这类关系型数据库的强项是：数据结构固定、表和表之间有明确关系、我需要 transaction（事务，一组操作要么全成功要么全失败，比如转账扣钱和加钱得绑一起）、需要强一致。像订单、账户余额、支付这种一分钱都不能错的，我会选 MySQL。

MongoDB 这类 NoSQL 的强项是：数据结构灵活、字段经常变、单条文档就能装下一大坨嵌套数据，而且天生好水平扩展、扛得住海量写入。像用户行为日志、feed 流内容、评论这种量大、结构又不太规整、也不需要跨表事务的，我会考虑 MongoDB。

基本取舍我会这么一句话总结：MySQL 用一致性和 join 能力换了扩展的难度，MongoDB 用灵活和扩展性换了强一致和复杂查询的能力。所以选型我看两点——数据关系强不强、要不要事务。要，就 MySQL；不要而且量特别大，就 NoSQL。面试时我甚至会说两个混用，核心账务走 MySQL、周边海量数据走 MongoDB，这在真实系统里很常见。

Sharding（分库分表）+ consistent hashing 一句话

当单台数据库存不下或者扛不住了，我就得做 sharding，也就是把一张大表的数据按某个 key（比如 user_id）拆开、分散到多台机器上，每台只存一部分。什么时候用？数据量或写入量大到单机顶不住的时候，加从库读写分离也救不了写入，那就只能拆。

最简单的拆法是按 user_id 取模（user_id 对机器数量取余数，余几就放第几台）。但这里有个大坑：一旦我要加机器、机器数量变了，取模的结果几乎全变，等于所有数据都得搬家，这叫 rehash 灾难。consistent hashing（一致性哈希）就是来解决这个的，我一句话讲清它：它把机器和数据都映射到一个环上，一个数据顺着环找到的下一台机器就归它管，这样加一台机器时只影响环上相邻的一小段数据、绝大部分数据不用动。所以面试里只要提到分片扩容，我就会把 consistent hashing 搬出来当答案。

消息队列（Kafka）：削峰、解耦、异步

Kafka 这类消息队列，我一般摆在生产者和消费者中间，生产者把消息扔进队列就不管了，消费者按自己的节奏慢慢取来处理。它主要解决三件事，我面试就记这三个词：削峰——流量突然暴涨（比如秒杀），请求先堆到队列里排队，后面的服务按自己能力慢慢消费，不至于被瞬间打死；解耦——上游服务不用知道下游是谁、也不用等下游处理完，两边通过队列隔开，一边挂了另一边还能继续；异步——像发通知、发邮件这种不用立刻返回结果的活儿，丢进队列让别人后台慢慢做，主流程赶紧把响应还给用户。

基本取舍是：加了消息队列，系统就从同步变成最终一致了，也就是这件事不是“立刻做完”而是“过一会儿会做完”。所以坑在于我得接受延迟、还得处理消息可能重复消费或者顺序问题。像“转账后立刻要看到余额”这种就不适合走队列，但“下单后发个短信”就很适合。

CDN（内容分发网络）

CDN 就是在全世界各地都放一堆缓存节点，把图片、视频、CSS、JS 这类静态资源提前分发到离用户近的节点上。什么时候用？我的服务有大量静态资源、而且用户分布在不同地区的时候。好处很直接：用户从最近的节点拿东西，延迟低，而且这些流量根本不用回到我的源站，源站压力小一大截。

基本取舍和坑跟 cache 是一类的：CDN 上的内容也可能是旧的，我更新了源站的图片，CDN 节点上可能还是老版本。所以实践里常用的办法是给静态文件名带上版本号或 hash（比如 main.a1b2c3.js），一改内容文件名就变、

URL 就变，CDN 自然当成新资源去拉。CDN 只适合放不常变的静态内容，动态的、每个用户都不一样的数据别往里塞。

主从复制与读写分离

这是给数据库扩读能力用的。主从复制就是搞一个 master（主库）负责写，再挂几个 slave（从库），master 把自己的数据变更同步复制到各个 slave 上。然后我做读写分离：所有写请求走 master，读请求分散到各个 slave。什么时候用？典型的读多写少场景，单个数据库读扛不住了，我加几个从库就能把读的压力分摊掉，比直接分片改动小得多。

基本取舍和坑就是主从延迟：master 写完到数据同步到 slave 之间有个时间差（可能几毫秒到几百毫秒）。这就导致一个经典问题——用户刚发了条评论（写进 master），紧接着刷新页面去读（读的是还没同步的 slave），结果发现自己的评论不见了。应对办法是对这种“写完马上要读”的场景强制走 master 读，其他不敏感的读才走 slave。面子里我会主动提这个延迟问题，显得考虑周全。

Bloom filter（布隆过滤器）

bloom filter 是一个特别省内存的小工具，它能快速回答“这个东西大概率不存在 / 可能存在”。原理不用深挖，我一句话说明白它的特点：它可能误判说“存在”（其实不存在），但绝不会误判说“不存在”（说不在就一定不在）。这个不对称是结构决定的、**反不过来**——插入时把几个哈希位置 1，位只增不减，所以“有一位是 0”能确定没插入过，“全是 1”却可能是别的元素凑巧覆盖的。想要反向的假阴性得换 Counting Bloom Filter 或 Cuckoo Filter，但那是为了支持删除才引入的副作用。什么时候用？最经典的就是配合前面讲的 cache 穿透——请求来了先问 bloom filter，它要是说“这个 key 肯定不存在”，我直接返回、连数据库都不用查，就把那些查不存在数据的恶意请求挡在门外了。爬虫去重、判断某 URL 是否爬过也常用它。

基本取舍就是它的这个“假阳性”特性：它省内存是靠牺牲精确度换来的，会有小概率把不存在的说成可能存在，这时候最多就是多查一次数据库、不会出错。而且标准的 bloom filter 只能加不能删。所以我用它的定位是“前置的粗筛快速挡一道”，不是拿它当精确的存在性判断。够挡掉大部分无效请求就行。

Part 3 — 高频题库

题 1 — TinyURL 短链系统

需求澄清

我拿到这题会先停一下，跟面试官确认清楚要做的边界，别上来就画框图。功能上我关心几件事：核心就是给一个长链接（比如一个很长的带一堆参数的 URL），系统生成一个短链接，用户点这个短链接能被重定向回原来的长链接。这是主线。然后我会问几个扩展点：短链要不要支持用户自定义别名（就是让用户自己指定 `tinyurl.com/mypromo` 这种）？短链要不要有过期时间？要不要做点击量统计、做分析（比如这条链接被点了多少次、来源是哪）？这些会影响我后面的数据模型，所以我要先问清楚哪些是 MVP 要的、哪些是加分项。

非功能性需求我会重点强调几点。第一是这是个典型的**读多写少**系统，创建短链是低频的，但点击跳转是高频的，读写比例可能是 100:1 甚至更悬殊，这个判断直接决定了我后面要重点上 cache。第二是**低延迟**，重定向必须快，用户点一下不能卡，几十毫秒内要跳走。第三是**高可用**，因为一旦这个服务挂了，所有已经发出去的短链就全废了，别人分享出去的链接全打不开，所以可用性比强一致性重要。第四是短链**不能冲突**，两个不同的长链接不能映射到同一个短码。

规模估算

我先假设一个量级，跟面试官对齐一下：假设每天新建 1000 万条短链（也就是每天 1 千万次写）。那写的 QPS = $10,000,000 / 86400$ （也就是每天的写请求数除以一天的总秒数 86400），算下来大概 115 QPS，写其实很低。读按 100:1 的读写比来算，读 QPS = 115×100 （也就是写的 QPS 乘以读写比 100），大概 11,500 QPS，峰值我一般再乘个 2 到 3 倍留余量，按 3 万 QPS 准备。

存储我这么估：假设系统跑 5 年，5 年总共存的短链数 = $10,000,000 \times 365 \times 5$ （也就是每天 1 千万条乘以一年 365 天再乘以 5 年），大概是 182 亿条。每条记录我粗估存 500 字节（短码、长链、创建时间、过期时间、用户 id 这些加起来），总存储 = $182 \text{ 亿} \times 500 \text{ 字节}$ （也就是记录条数乘以每条大小），大概是 9 TB 左右。这个量级单机放不下，但也没夸张到需要几百台机器，我心里有个数，后面数据层要做 sharding。

再算一个关键的——**短码要多长**。我用 base62 编码（就是用 0-9、a-z、A-Z 这 62 个字符来表示，比只用数字的 10 进制紧凑很多）。如果短码取 7 位，能表示的组合数 = 62 的 7 次方（也就是 62 自己乘 7 次），大约是 3.5 万亿，远远够装我上面算的 182 亿条，所以 7 位就够了，我会用 6 到 7 位。

API 设计

我设计两个核心接口。第一个是创建短链，`POST /api/v1/urls`，请求体里带长链接 `long_url`，可选带自定义别名 `custom_alias` 和过期时间 `expire_at`，返回生成的短链 `short_url`。第二个是重定向，`GET /{short_code}`，这个不是我们自己定义的 REST 接口，而是浏览器直接访问短链时打过来的，服务端查到对应长链后返回一个 302 重定向。

这里有个我会主动讲的取舍：重定向用 **301 还是 302**。301 是永久重定向，浏览器会缓存，下次用户再点同一个短链，浏览器直接跳过去都不打我们服务器了——好处是省流量，坏处是我们**统计不到点击量**，而且一旦这条映射要改就麻烦了因为浏览器已经缓存死了。302 是临时重定向，每次都会回到我们服务器，虽然多了一跳，但我们能记录每次点击。所以如果需求要做点击统计，我就用 302；纯粹追求性能不管统计就用 301。我一般默认选 302，因为分析数据通常是有价值的。

数据模型

这题的数据其实很简单，就是一张核心表，字段是短码 `short_code`（做主键或者唯一索引）、长链 `long_url`、创建时间、过期时间、创建者 id、点击计数这些。关键是选什么库。

因为这个访问模式非常干净——绝大部分操作就是「拿短码查长链」这种主键点查，没什么复杂的关联查询和事务，而且数据量大到要 sharding，所以我倾向用 **NoSQL**，比如 key-value 存储或者像 MongoDB / Cassandra 这类，key 就是短码、value 是长链和元数据，天然适合按 key 水平扩展。如果面试官更熟悉关系型，用 MySQL 也完全可以，就是后面要自己做 sharding（比如按短码哈希分片）。我会说清楚：选 NoSQL 主要是图它水平扩展和高吞吐点查方便，而不是因为我们有什么复杂结构。读多写少 + 主键点查的场景，cache 加 NoSQL 这套组合是最省心的。

高层架构

我把数据流从头走一遍。用户创建短链时，请求先打到 load balancer，分发到某台应用服务器；应用服务器负责生成一个唯一的短码，然后把「短码 → 长链」这条映射写进数据库，再把短链拼好返回给用户。

用户点击短链跳转时，请求同样先过 load balancer 到应用服务器；应用服务器拿短码**先查 cache**（比如 Redis），因为读多写少、而且总有些热门链接被反复点，cache 命中率会很高，命中就直接拿到长链返回 302，根本不碰数据库；如果 cache 没命中，才去数据库查，查到之后回填进 cache（设个过期时间），再返回 302 重定向。这样绝大部分读请求都被 Redis 挡住了，数据库压力很小。

再往大了扩，读侧我还可以在最前面挂 CDN 或者边缘节点，把最热的短链映射缓存到离用户更近的地方，进一步降延迟。

生成短码怎么做，这是这题的核心难点，我一般讲两条路线。第一条是**哈希法**：把长链做一次 hash（比如 MD5），取前几位再 base62 编码当短码。好处是无状态、简单；坏处是哈希会**碰撞**，两个不同长链可能算出同一个短码，得处理冲突——通常做法是生成后先查库，撞了就加盐或者取不同位再试一次。第二条是**发号器法**：维护一个全局自增的 ID，每来一个请求分一个唯一的数字 id，再把这个 id 用 base62 编码成短码。好处是**天然不冲突**，因为 id 本身就唯一；难点是这个全局自增在分布式下不好做，单点发号会成瓶颈。解决办法是搞一个专门的发号服务，或者预先把 id 空间切成一段一段（比如给每台机器分配一个号段，机器在自己号段里自增，用完再领下一段），这样各台机器互不打架也不用每次都去抢锁。我个人偏好发号器 + 号段这套，因为它彻底绕开了碰撞问题，代价是引入了一个要维护的发号组件。

关键取舍与瓶颈

第一个是**301 vs 302** 前面说过了，本质是「省性能」和「能统计」之间选，我默认 302 保住分析能力。

第二个是**哈希 vs 发号器**，哈希简单无状态但要处理碰撞，发号器不碰撞但要维护全局唯一 id 且分布式下有单点风险，我倾向号段方案来化解这个瓶颈。

第三个是**cache 的分量**，这题的性能命脉就在 cache 上。因为读远大于写、又存在热点链接，Redis 命中率能做得很高，数据库基本只承接创建和 cache miss。这里要注意 cache 和数据库的一致性，但短链有个好处——映射**基本不会变**（一条短链创建后长链几乎不改），所以一致性问题比一般系统轻很多，我用「读时回填 + 设过期」就够了，不用搞复杂的双写一致。

第四个是**数据库扩展**，9 TB 单机放不下，得 sharding。用 NoSQL 的话按短码做 key 天然能水平分；用 MySQL 就得手动按短码 hash 分库分表。这块我会说是随数据增长自然要走的一步。

第五个是**可用性**，因为发出去的短链是永久资产，服务不能挂，所以每层都要冗余——应用服务器多实例无状态、load balancer 后面挂一堆、数据库要有副本、Redis 也做主从或集群。整个系统我会优先保可用性和低延迟，短链这种场景没必要追求强一致。

题 2 — Rate Limiter 限流器

需求澄清

拿到这题我一般先别急着讲算法，先跟面试官对齐我们到底在限什么、限成什么样。功能层面我会问：这个限流器是限谁？是按用户 ID 限、按 IP 限、还是按 API key 限？限制规则长什么样，比如“每个用户每分钟最多 100 次请求”这种。超了之后怎么办，是直接拒绝返回 HTTP 429（也就是 Too Many Requests 这个状态码），还是排队等一会儿。规则是不是要能配置、能热更新，不用重新发布服务就能改阈值。

非功能层面我会强调几点：一个是低延迟，因为限流器是挡在所有请求前面的，它自己不能成为瓶颈，我心里的预期是它加的延迟得在毫秒级甚至亚毫秒级；第二是高可用，这东西挂了会很尴尬，所以后面我会专门讲一个 fail-open 还是 fail-closed 的取舍；第三是准确性，分布式环境下多台机器一起数数，能不能数准，这块是这题的核心难点。我会跟面试官说清楚我们通常接受“大致准确”，不追求一个都不多放。这块我在 Newsbreak 做 UGC anti-abuse 的时候就是这个思路，防刷那种场景稍微多放几个不致命，但延迟和可用性绝对不能牺牲，所以是 fail-open。（真被追问可以讲那次误伤内部脚本、后来改成只采集不拦截的教训：有误判率的判断不该有拦截权，有拦截权的规则必须是确定性的。）

规模估算

假设我们是一个中等规模的服务，日请求量 10 亿。那平均 QPS = $1,000,000,000 / 86400$ （也就是十亿次请求除以一天有多少秒），算出来大概是 11,500 QPS。但限流器最怕的是峰值不是平均值，所以我一般按峰值是平均的 5 倍来预留，也就是峰值 QPS $\approx 11,500 \times 5 \approx 57,500$ （平均每秒请求数乘以五倍峰值系数），差不多小六万的量级。

存储这块我算一下：假设我们要给每个活跃用户维护一个计数器，活跃用户 1000 万，每条计数记录我就存一个 key 加一个整数计数加过期时间，算它 100 字节。那总内存 = $10,000,000 \times 100$ 字节 = 1 GB（一千万用户乘以每条一百字节，等于十亿字节，约一个 G）。这个量级单台 Redis 内存完全放得下，而且这些计数都是短命的、带过期时间的，所以我不需要持久化存储，放内存里就行。这个估算的结论是：这题的瓶颈不在存储容量，而在于每次请求都要读写一次计数器，也就是那接近六万 QPS 的读写压力，得让 Redis 扛得住、延迟够低。

API 设计

限流器对外其实就一个核心动作，我一般设计成一个内部方法或者一个中间件调用，签名类似

```
allowRequest(userId, apiName)
```

（传入用户 ID 和要访问的 API 名字，返回允许还是拒绝）。返回值我会带上几个信息：一个布尔值表示允不允许，还有剩余额度和多久之后重置。对应到 HTTP 层，如果被限了我会返回 429，然后在响应头里带上 X-RateLimit-Remaining 和 Retry-After（告诉客户端还剩多少额度、多少秒后

可以重试），这样客户端就知道该退避多久，不会傻乎乎地继续猛打。规则的配置我会另做一套接口或者放配置中心，跟运行时的判断路径分开。

数据模型

我要存的东西其实很简单，就是“某个 key 在某个时间窗口里已经用了多少次”。key 我一般拼成

`userId:apiName:窗口标识` 这种形式（用户加接口加时间窗拼成一个字符串做键），value 就是计数。存哪、用什么库这块我会直接选 Redis，理由有几个：一是它是内存数据库，读写都是亚毫秒级，正好满足限流器低延迟的要求；二是它天然带 TTL（也就是 key 的过期时间），我给每个计数器设个过期，窗口一过 key 自动消失，不用我自己写清理逻辑；三是它有原子操作 `INCR`（原子自增命令，多个请求同时打进来也不会数错），还能用 Lua 脚本把“读计数、判断、加一”几步打包成一个原子操作，避免并发下的竞态。我不会用 MySQL 这种磁盘库来存计数，太慢，扛不住那个 QPS。

高层架构

数据怎么流我从头讲一遍。请求先打到 load balancer，然后进到我们的限流器，这个限流器我一般做成一个中间件，挂在 API gateway 上或者作为服务前面的一层。请求进来，中间件先根据用户 ID 和接口名拼出 key，去 Redis 里对这个 key 做一次原子自增，拿到当前窗口的计数值。拿到之后跟规则里的阈值比一下：没超就放行，请求继续往后走到真正的业务服务；超了当场返回 429，根本不打到后端，这样后端也被保护了。规则本身放在一个配置中心或者单独的规则存储里，中间件启动时加载、之后定期刷新或者监听变更，这样改阈值不用重启。

这里的关键点是为什么要用一个集中的 Redis 而不是每台机器自己在内存里数。因为我们后端是多台机器、前面有 load balancer 做负载均衡，同一个用户的请求会被打到不同机器上。如果每台机器各数各的，比如我有 10 台机器、每台限 100 次，那这个用户实际能打 1000 次，限流就形同虚设。所以我把计数收敛到一个共享的 Redis 上，所有机器都往同一个计数器上加，这才数得准。这就是分布式限流的核心。

关键取舍与瓶颈

第一点是算法的取舍，这是这题最经典的部分。最简单的是 fixed window（固定窗口），就是按整分钟切，每分钟一个计数器，实现最简单，但有个边界问题：如果限每分钟 100 次，用户在第 59 秒打 100 次、下一分钟第 1 秒又打 100 次，那在这跨越窗口的两秒里他实际打了 200 次，把限制翻倍了。为了解决这个我会讲 sliding window（滑动窗口），它不按死板的整分钟切，而是看“当前时刻往前推一分钟”这个滚动区间里的量，边界平滑很多，代价是要多存一些数据、算起来稍微复杂。然后是 token bucket（令牌桶），桶里按固定速率放令牌、每个请求取一个令牌，桶还能攒一定量的令牌，所以它允许短时突发，平时不打的时候攒着、突然来一波能放过去，这个在实际里用得非常多。最后是 leaky bucket（漏桶），请求进桶、以固定速率漏出去处理，它是强行把流量整平、不允许突发，适合保护下游那种吞吐固定的场景。我一般的取舍是：要平滑限速、又想容忍正常的突发流量，就选 token bucket；要严格保护下游、把速率压死，就用 leaky bucket。我在 Newsbreak 做防刷的时候更接近 sliding window 加计数那套，因为反作弊场景我关心的是一段滚动时间内的总量，不太需要容忍突发。

第二点是原子性，也就是并发下怎么数准。同一个用户的两个请求几乎同时打到 Redis，如果我先读计数、判断、再写回，中间就有竞态，可能两个都读到 99、都以为没超、都放行。解决办法就是用 Redis 的原子命令或者 Lua 脚本，把这几步做成一个不可分割的操作，这块我一定会跟面试官点明。

第三点是 fail-open 还是 fail-closed，就是 Redis 挂了或者超时了怎么办。fail-open 是限流器出问题默认放行，好处是不影响正常用户、服务照常跑，坏处是这时候限流失效、真有人刷你也挡不住。fail-closed 是出问题默认拒绝，安全但会误伤大量正常用户、可能引发雪崩。这个没有标准答案，得看场景：如果限流主要是为了保护后端别被打垮，我倾向 fail-open，毕竟不能因为限流器自己挂了就把整个服务也拖死；如果限流是安全防线、比如防支付被刷，那可能就得 fail-closed。我在 anti-abuse 那边是偏 fail-open 的，因为可用性优先，少防几个可以事后补。

第四点是延迟与准确的取舍。每个请求都去打一次共享 Redis，会引入一次网络往返。如果嫌这个延迟高、或者 Redis 压力大，可以做本地计数加定期同步：每台机器先在自己内存里数，再周期性地汇总到 Redis，这样快很多，但短时间内会数得没那么准。这就是拿准确性换延迟和吞吐，看业务能不能接受一点点误差。

第五点是扩展性，也就是当单台 Redis 扛不住那接近六万 QPS 的时候怎么办。我会做 Redis 的 sharding（分片），按用户 ID 做 hash 把不同用户的计数分到不同 Redis 节点上（用一致性哈希 consistent hashing 来分，加减节点时挪动

的 key 少)。因为限流是按 key 独立计数的,天然好分片,同一个用户的 key 永远落到同一个节点,不会跨节点算,所以水平扩展起来很顺。另外热点用户可能让某个节点变热,这时候可以对超级大 key 单独处理,但 entry-level 层面知道"用 sharding 横向扩、按 key 分"这个思路就够了。

题 3 — 设计 Twitter (关注流 feed)

需求澄清

我上来会先跟面试官对齐我们到底在设计什么样的 feed,因为这个词底下其实是两套完全不同的架构。一种是**关注流**(Twitter 的 Following、Facebook 的 News Feed),内容池由关注关系严格定义,核心难点是怎么把几百个关注对象的内容高效聚合起来——也就是 fanout 的写扩散读扩散取舍。另一种是**推荐流**(Twitter 的 For You、TikTok 的 For You),内容池是全站,核心难点是召回排序漏斗,fanout 那套完全用不上。我会问一句:"我们这版聚焦在关注流还是推荐流?"如果面试官让我选,我会选关注流,因为它是这题的经典考点。

下面按**关注流**展开。功能需求我会锁这么几条:用户能发帖;用户能刷自己的 home timeline,也就是所关注的人发的内容聚合成的流;关注/取关;feed 支持无限往下翻,一次拉一批。

排序上我先**假设纯时间倒序**,这样能把 fanout 那套核心取舍讲清楚;真实产品是按 ranking 打分排的,这个我后面深挖时单独展开,它影响读路径但不影响 fanout 这一层。

我会主动排除掉两块:一是**点赞、收藏、转发、评论**这些互动功能——存储上就是关联表加索引,计数用 Redis 累加异步刷库,架构上没什么取舍;而且 Twitter 的回复本身就是一条带 in_reply_to 指针的 tweet,复用同一套 post 和 fanout 流程,不需要独立的评论系统。二是**点进某人主页看他自己发的帖**那条 timeline,它就是按 user_id 查一次表,不是难点。真正难的是 home timeline,要跨几百个关注对象聚合,这才是 fanout 要解决的问题。

非功能需求这块我觉得对 feed 是重点,我会强调三个:一是低延迟,用户刷新最好 200 毫秒以内出结果,不能让人等;二是高可用,feed 挂了整个 App 就白屏了,可用性要比"发帖成功"更重要——刷不出旧内容比发不出新帖子严重得多;三是最终一致就行,我关注的人发了帖,晚个几秒钟出现在我 feed 里完全能接受,feed 天然不需要强一致。

规模估算

估算之前我会先说明一件事:**这题要区分两个用户口径**。数据库按**总注册用户和累积数据量**算,因为不活跃用户的历史帖子也得留着;Redis 的 inbox 按**活跃用户**算,因为不来读的人不需要给他维护 inbox。这两个数差好几倍,混用会估错。

先定假设:

```
总注册用户 = 5000 万
DAU = 1000 万
每天新增 tweet = 500 万条 (人均 0.5 条/天)
读: 每人每天看 1 次自己的 timeline + 访问 3 个别人的页面, 每页 10 条
人均关注 200 人
```

QPS:

```
写 = 500万 ÷ 86400 ≈ 58 QPS
读 = 1000万 × (1 + 3) × 10 = 4 亿次浏览/天
    = 4亿 ÷ 86400 ≈ 4,630 QPS
峰值按 3 倍算 ≈ 1.4 万 QPS
读写比 = 4亿 : 500万 = 80:1
```

这里我要强调**读的口径不能只算 timeline**——用户还会点进别人主页、点开单条推文看详情,这些都是读。只算 timeline 会把读写比严重低估。80:1 这个数量级跟 Twitter 公开过的比例 (大约 100:1) 是对得上的。

这个判断很关键——它直接决定了后面愿意在写的时候多做点事来把读做快，也就是 push 模型的直觉来源。

post 存储（按累积算）：

一条 tweet 连元数据算 10KB（正文才 280 字符，但要带 id、作者、时间戳、计数、索引开销这些；图片视频走对象存储只存 URL）
每天 = 500万 × 10KB = 50 GB/天
一年 ≈ 18 TB，五年 ≈ 90 TB

关注关系（按总用户算）：

5000万 用户 × 人均 200 关注 = 100 亿行 × 约 16 字节 ≈ 160 GB

inbox (Redis) ——这块必须单独估，而且口径不一样：

每个 inbox 条目我不存完整内容，只存定位信息：tweet_id 8 字节 + author_id 8 字节 + 几个标记位 1 字节，大概 **17 字节一条**。带 author_id 是为了读的时候能并发查 post 内容和作者信息，省一轮串行往返。

inbox 按 TTL 窗口内活跃过的人算。TTL 设 30 天，对应的是月活不是日活。
MAU 通常是 DAU 的 2-3 倍，1000 万 DAU 取 2500 万 MAU。

2500万 × 每人最多 800 条 × 17 字节 ≈ 340 GB 裸数据
Redis 实际占用（对象头、跳表指针、分配器对齐）按 2-3 倍算 ≈ 700GB - 1TB

这个数要换算成机器数才有意义，我会顺手算一下：

按单机 64GB 内存、留 70% 可用（要给 fork、内存碎片留 buffer）：
每台可用 ~45GB
1TB ÷ 45GB ≈ 23 个分片
加一主一从做高可用 → 约 46 台

所以 inbox 大概是二三十个分片、四五十台机器的 Redis 集群，按 userId 分片。把 TB 数换算成机器数很重要——它让“这块贵”变成一个具体的、可以讨论的数字。

而且这两个参数是**可调的旋钮**：如果面试官说机器预算只有 10 台，我第一个动的就是 TTL（30 天砍到 7 天）和每人条数（800 砍到 200），内存立刻降到一两百 GB。TTL 短、条数少省内存但重建更频繁，反过来省重建但费内存——这是个明确的取舍，不是拍死的数。

对比一下：post 存储一年 18TB 在磁盘上，inbox 稳态 1TB 在内存里。**inbox 的绝对量小一个数量级，但它是最贵的那块**——内存每 GB 成本比磁盘高一两个数量级，算下来花的钱是同量级的。所以后面所有关于 inbox 的优化（TTL、长度截断、跳过不活跃粉丝、大 V 不 fanout）都是在省这份钱。**API 设计**

我会设计得很薄，主要就三个。发帖是 POST /v1/posts，body 里带内容，用户身份从 token 里拿，不放在 body 里（安全考虑）。拉 feed 是 GET /v1/feed?limit=20&cursor=xxx ——这里我特意用 cursor 而不是 page number。为什么？因为 feed 一直在变，用 offset 分页翻页会重复或者漏内容（前面插入了新内容，整体往后挪，你按 offset 取下一页会拿到刚看过的）；而且 offset 越深数据库要扫过并丢弃的行越多，深翻页是线性退化的。cursor（上一批最后一条的 id 或时间戳）记的是绝对位置，不受插入删除影响，翻到多深都是一次索引定位。代价是只能顺序翻不能跳页，但 feed 本来也不需要跳页。关注是 POST /v1/follow，body 里带被关注的 userId。

返回体我会让 feed 接口直接返回**一批完整的、能直接渲染的对象**加上 next_cursor，不是一串 id。这里 limit 我要说明一下——它是一个**批次而不是一屏**。客户端一次拉 20-30 条，用户往下滑到还剩几条时后台静默预取下一批，这样滑动是无缝的，不会每翻一屏就转圈。批次大小是个权衡：太小预取窗口不够、请求太频繁；太大首屏慢、用户提

前退出的浪费也大。而且批次越大，两次请求间隔越长、feed 变动越多，这也是必须用 cursor 而不是 offset 的另一个原因。

数据模型

Post 存储我会选 MongoDB，理由跟前面估的量级直接挂钩：一年 18TB，MongoDB 分片存完全够，而且原生 sharding 不用自己搭中间件；写入才 58 QPS，压力很小；查询模式就是按 tweet_id 点查和按 user_id 取最近若干条，不需要 join 和跨文档事务。

如果面试官问“为什么不用 Cassandra”，我会说：Cassandra 的强项是无主架构带来的超高写入吞吐和多机房地本地写，但这题写入只有 58 QPS，那个能力用不上，反而要付出查询能力极弱的代价——Cassandra 基本只能按 partition key 查，一个查询模式就得单独建一张表、数据反规范化存好几份。如果日活再大一两个数量级、写入到几万 QPS，那 Cassandra 就是更好的选择。**选型看的是写入量级和查询模式，不是数据总量。**

User 表放什么都行，它 schema 稳定、量小（5000 万行），MySQL 也完全够。Follow 关系单独存，一张（follower_id, followee_id）的表加两个方向的索引就够——正向查“我关注了哪些人”，反向查“谁关注了我”（fanout 时要用）。100 亿行大概 160GB，分片存没问题。量再大它本质是个图，但这题一张带索引的表足够，不需要专门上图数据库。

然后是最关键的一份数据——**inbox**（也叫 feed cache / home timeline），放 Redis 集群按 userId 分片。

数据结构我用 **Sorted Set**，不用 List：

```
ZADD "inbox:{userid}" {发布时间戳} {tweet_id + author_id + flags}
ZREVRANGE "inbox:{userid}" 0 29          # 取最新 30 条
ZREMRANGEBYRANK "inbox:{userid}" 0 -801  # 只保留最新 800 条
EXPIRE "inbox:{userid}" 2592000          # 30 天 TTL，用户读 feed 时刷新
```

选 Sorted Set 而不是 List 的理由有两个：一是**翻页**，cursor 就是时间戳，Sorted Set 能直接按 score 范围取，List 只能按下标翻、等于退回 offset 分页；二是**幂等**，member 唯一，fanout 重试时同一条重复写入不会产生重复项，删帖也能直接按 id 删。至于插入是 $O(\log n)$ 而不是 $O(1)$ ，在 800 条这个量级就是十次跳转，完全被网络往返淹没，不构成选型理由。

存什么：只存 tweet_id + author_id + 标记位，**不存正文、作者昵称这些展示内容**。三个原因：内容存一份 vs 存几十万份，内存差几十倍；tweet 被编辑、删除、点赞数变化时，散在几十万个 inbox 里根本没法同步；内容有独立的一层缓存，key 是 tweet_id，所有人共享同一份。author_id 是例外——它永远不变，冗余进来是为了省一轮串行查询，不是内容缓存。

TTL 必须设。不设的话 key 会随注册用户数无限累积，5000 万注册用户就是 5000 万个 key 永久占内存。设 30 天、用户每次读 feed 时刷新，内存就自动收敛到“最近 30 天活跃过的用户”。而且这个 TTL 顺带解决了另一个问题——fanout 时先 EXISTS 判断一下，key 不存在说明这人长期没来，直接跳过不写，**写放大能砍掉一大截**。

注意 TTL 和长度截断是**两个独立维度**：TTL 控制 key 的存活（有多少个 inbox），ZREMRANGEBYRANK 控制单个 key 的大小（每个 inbox 多少条），两个都要做。

高层架构

写路径：用户发帖，请求进 load balancer 到 Post Service，先把 post 落到 post 存储（source of truth，必须先持久化）。落库成功后，Post Service 往 Kafka 扔一个“有新帖”的事件，然后就给用户返回成功了——**同步请求里不做 fanout**，扇出交给异步。后面 Fanout Service 消费 Kafka，查发帖人的粉丝列表，把这条写进每个粉丝的 inbox。这就是 fanout on write。

读路径就很轻：请求到 Feed Service，三步——先 ZREVRANGE 从 Redis 拿 30 条 (tweet_id, author_id)，然后**并发**去批量取 post 内容和作者信息（各走各的缓存，miss 才回库），最后拼装成完整对象返回。因为写的时候已经把 inbox 预计算好了，读就是一次 Redis 加两次并发批量查。

这里有一点要说清楚：**id list 只在服务端内部流转，客户端拿到的是能直接渲染的完整对象**。不会让客户端拿一串 id 再自己去查——那样多一轮公网往返（几十上百毫秒），而服务端到 Redis/DB 是内网（几毫秒），差一个数量级，而且会把存储结构暴露给客户端。

至于返回**全文还是摘要**，取决于内容体积。Twitter 的推文才 280 字符，**timeline** 直接返回全文没压力（官方 API 默认就返回完整 text）；但如果是长文、视频这类重内容，就该只返回摘要、点开再取详情——判断标准是内容大小乘以未点击率，用户只点开 5-10%，内容越大浪费越多。另外即便返回了全文，点开详情页通常还是要发一次请求，因为详情页有回复列表、conversation 这些 timeline 里不该带的东西。

关键取舍与瓶颈

第一，push 还是 pull，这是这题的灵魂。刚才讲的是 push (fanout on write)，优点是读极快，缺点是写放大严重——一个人发帖要写进所有粉丝的 inbox。反过来 pull (fanout on read) 是发帖啥都不做，用户刷 feed 时现去把关注的所有人的最新 post 捞出来合并排序。pull 写便宜、不浪费存储，但读的时候现算，关注几百上千人时延迟很高。取舍就是：push 拿存储和写成本换读延迟，pull 反过来。因为 feed 读多写少（前面算的 100:1），默认偏 push。

第二，大 V 问题 (celebrity / hot key)，这是纯 push 的死穴。一个明星几千万粉丝，发一条帖 fanout 要写几千万次，写放大直接爆炸，而且很多粉丝当天根本不上线，白写了。业界标准答案是**混合模型**：普通用户走 push，大 V 走 pull——他的帖子不扇出，等粉丝读 feed 时再单独把他关注的这几个大 V 的最新帖捞出来，跟 push 预存的那部分合并。阈值参考 Twitter 的做法大概是**粉丝数一万**以上归到 pull（具体数字各家不同，重点是讲清楚为什么要分档）。

这里顺带说 **outbox**（用户自己发的帖）。默认我不专门维护——点进某人主页看他发的帖，直接 WHERE user_id = ? ORDER BY created_at DESC 查库，有索引就是一次范围扫描，专门存一份反而多个一致性问题。唯一值得上 Redis 的是混合模型里大 V 那条 pull 路径：每次有粉丝刷 feed 都要现拉大 V 的最新帖，同一个大 V 会被海量粉丝反复读，这是典型热点。所以只给大 V 缓存一份，5-10 分钟 TTL，miss 了回表。

inbox 和 outbox 的区别值得点一句：**inbox 是必须的物化视图**（不预计算就没法快速读 feed），**outbox 只是热点缓存**（不存也能回表查）。判断标准是丢了的代价——inbox 丢了要遍历几百个关注对象重建，outbox 丢了就一次索引查询。两者必要性完全不同，别当成对称的东西。

第三，inbox 的内存控制。前面估过大概 1TB、二三十个分片，这是最贵的一块，所以省内存的手段要一起上：TTL 30 天加读时刷新让内存收敛到活跃用户；fanout 时 EXISTS 跳过不活跃粉丝；长度截断在 800 条，更深的翻页走 pull 兜底（几乎没人翻那么深）；大 V 不 fanout。这几条都是在压前面那个 1TB 的数。

第四，post 存储怎么分片，这块有三个选择，各有各的问题：按 **userID** 分片，同一个人的 tweet 都在一起、查他的主页很快，但大 V 会造成数据倾斜和热点；按 **tweetID** 分片（哈希打散），分布最均匀，但“查某个用户最近的 tweet”要问所有分片再归并，延迟高；按**创建时间**分片，取最近内容很快，但流量会全部压在最新的那个分片上，老分片闲置。我会选 **userID 分片配合大 V 特殊处理**——因为主要查询模式就是按用户取，倾斜问题正好可以复用前面混合模型里对大 V 的识别逻辑。

第五，降级兜底，这块我会主动讲，因为关注流有个特殊约束。Ranking 服务挂了最好办——inbox 本来就按时间戳存的，直接按时间倒序返回就是个完全可用的 feed，只是质量差一点，这是关注流相比推荐流的结构优势，它有个天然的兜底顺序。inbox 读失败就退回 pull 现算一次，慢但结果是对的，这条路径本来就要为不活跃用户重建而存在，正好复用。但如果 Redis 大面积失效，pull 兜底也扛不住——所有用户一起走 pull 会把数据库打爆，这时候只能限流加返回旧数据。

有一点我想强调：**关注流不能降级成推荐内容**。推荐流挂了可以拿热门内容垫底，用户感知不到；但关注流的产品契约是“我关注的你得给我看”，掺进没关注的内容就是破坏契约。所以它的降级方向只能是质量和新鲜度，不能是内容范围。

第六，ranking。前面为了讲清楚我按时间序说的，但真实产品是按打分排——用相关性、新鲜度、互动预测这些信号给候选打分。我会把 ranking 拆成独立阶段：先召回（从 inbox 拿候选），再排序，最后截取一批返回。好处是召回和排序解耦，模型可以独立迭代不影响存储层。

这里有个容易搞混的点：**打分是读的时候算的，不是写的时候**。Sorted Set 里那个 score 是发布时间戳，只负责保证“能高效取到最近的一批候选”，不是 ranking 分数。真正的排序发生在读路径——从 inbox 取比展示条数多得多的候选（比如展示 30 条就取 500 条），实时打分再取 top 30。为什么不能在写入时打分？因为分数**因人而异**（同一条帖子对不同粉丝的相关性完全不同，发帖那一刻不知道要给谁看），而且会随时间衰减、随互动数变化，写入时算的分几分钟就过时了。

顺带一提，inbox 的顺序虽然会被 ranking 覆盖，但它依然要有序——因为“取最近 500 条”这个动作本身依赖顺序。**召回决定了排序的天花板**，没被召回的内容排序再好也出不来。

再往上扩展可以提：ranking 分粗排精排两阶段、去重、过滤已读。

如果面试官要的是推荐流 (For You)

这时候 fanout / inbox 那套完全不适用——推荐结果没法预计算，因为它的价值就在于用上最新的行为信号。架构变成一条多阶段调用链：多路召回（关注的人、协同过滤、内容相似、热门、地理位置）→ 粗排砍量 → 精排打分 → 混排控多样性 → 返回。关注内容在这里只是召回的一路候选，不独占。

系统设计的考点也跟着变，我会往这四个方向讲：**延迟预算**怎么分配给各阶段；多路召回**能不能并发**、某一路超时怎么办（丢掉这一路继续，不整体失败）；推荐服务整体挂了的**降级兜底**（热门内容垫底，绝不能白屏）；以及**什么能缓存什么不能**——结果不能缓存，但用户画像、内容特征、内容详情都可以，这个区分本身就是加分点。排序模型的算法细节不是系统设计的考点，可以明确说“那部分我当黑盒服务调用”。

收尾

如果面试官问扩展性，瓶颈主要在三处：inbox 那 1TB 内存、四五十台机器是最硬的一块，靠 Redis 集群按 userId sharding 水平拆开；Fanout Service 无状态，靠 Kafka 削峰加消费者水平扩；post 存储读压力大就加读副本和多级 cache。整体思路始终是那句——读多写少，把复杂度往写侧和异步侧推，让读路径尽量只碰内存。

如果还有时间，我会用一句话讲演化方向：**一旦 Following 也引入 ranking，它和推荐流的架构就趋同了**——都变成召回加排序，inbox 从“feed 的答案”退化成“其中一路召回源”。所以真实产品通常会保留一个纯时间序、不做 ranking 的 Following tab，因为关注这个动作的产品契约就是“我关注的你得给我看”，算法一介入这个承诺就破了。这也是为什么 Twitter 现在把 For You 和 Following 做成两个独立 tab。

题 4 — Notification / Push 通知系统

需求澄清

我拿到这道题不会马上开始画架构，我会先跟面试官对一下我们到底要做什么。通知系统这个词很宽，我想先确认几件事。

功能上我会问：我们要支持哪几种通知渠道？一般来说至少是三种——手机的 push 通知（走 APNs 和 FCM）、短信（走 Twilio 这种）、还有邮件。我先假设重点是手机 push，因为这是最典型也最难的一块。然后我会问，通知是谁触发的？是别的业务后台服务调我们（比如“你关注的人发了新帖”这种），还是运营在后台圈一批用户群发（比如营销 push）。这两个场景的量级差很多，我下面估算会分开算。再一个我要确认的是模板——大部分通知不是随便发一段文字，而是“标题+正文+跳转链接”这种有固定格式的东西，同一个通知类型在不同语言、不同 A/B 分组下文案不一样，所以我会假设系统需要一套模板机制，调用方只传一个模板 ID 加几个变量，不传整段文案。还有就是用户偏好，用户能不能关掉某一类通知、能不能设置免打扰时段，这个我会当成必须有的功能。

非功能上我最关心几点。第一是**送达但可以有延迟**——通知系统通常不要求强一致，晚几秒到没关系，但绝对不能重复发（用户收到两条一模一样的 push 体验很差），所以去重和幂等是硬要求。第二是**高可用**，触发通知的上游服务不该因为我们挂了就被拖住，所以我们之间要解耦。第三是**削峰**，营销 push 经常是一瞬间要发几百万条，但下游 APNs/FCM 有速率限制，我们得能把这个尖峰压平。第四是可观测，每条通知发出去了没、被第三方拒了没、用户点没点，这些我都要能查到。

规模估算

我先拍个体量。假设我们有一亿日活用户（DAU = 100M，也就是每天有一亿个活跃用户），平均每个用户每天收到 5 条通知，那每天的通知量就是 $100M \times 5 = 500M$ （一亿乘以五等于五亿条每天）。

换算成平均 QPS，就是 $QPS = 500M / 86400 \approx 5800$ （也就是五亿条除以一天的八万六千四百秒，约等于每秒五千八百条）。但平均值没意义，因为营销 push 是尖峰，我会假设峰值是平均的 10 倍到 20 倍，也就是峰值 $QPS \approx 5800 \times 20 \approx 116000$ （约等于每秒十一万条）。这个尖峰正好说明为什么中间必须有一层消息队列来削峰。

存储上我算两块。一块是设备 token 表，每个用户可能有多台设备，假设平均 1.5 个 device token，那就是 $100M \times 1.5 = 150M$ 条（一亿五千万条 token 记录），每条记录 token 加上平台加上用户 ID 大概几百字节，整体也就几十 GB 到上百 GB，不大。另一块是通知发送记录，如果每条都要留存做审计，每天 500M 条，每条假设 200 字节，就是 $500M \times 200B = 100GB/\text{天}$ （每天大约一百 GB）。这个量一个月就是 3TB 左右，所以我会说发送日志只热存最近几天，老的滚到冷存储或者干脆按 TTL 过期。

API 设计

对上游我会暴露一个很简单的接口，让业务方不用关心底层是 APNs 还是 FCM。核心就一个 POST `/v1/notifications`，body 里带 `user_id`（发给谁）、`template_id`（用哪个模板）、`variables`（模板里要填的变量，比如发帖人的名字）、`channel`（push/sms/email，可以不转让系统按用户偏好决定）、还有一个非常关键的 `idempotency_key`（幂等键）。这个幂等键是调用方生成的，同一个逻辑事件重试时带同一个 key，我们服务端就能识别出“这条我处理过了”，直接返回上次的结果而不是再发一遍。

群发我会单独给一个 POST `/v1/campaigns`，接受一个用户圈选条件或者用户 ID 列表加模板，异步返回一个 `campaign_id`，然后有 GET `/v1/campaigns/{id}` 查进度。群发和单发分开是因为群发要走批处理、要更强的限速，跟实时单发的路径不一样。

再配套两个：GET `/v1/users/{id}/preferences` 和对应的 PUT，让用户读写自己的通知偏好和免打扰设置；还有一个给客户端上报 device token 的 POST `/v1/devices`，App 启动拿到 APNs/FCM token 后调这个把 token 存下来。

数据模型

我会分几张表，放在不同的库里，按访问模式选。

device token 表，我会放在一个 key-value 型或者文档型的库里，比如用 MongoDB 或者直接 Redis 做缓存层。它的访问模式几乎全是“给我 user_id，返回他所有的 device token”，是主键点查，没什么复杂关联，所以不需要关系型库。字段就是 user_id、token、platform（iOS 还是 Android）、last_active、还有一个是否失效的标记。这里有个坑是 token 会失效——用户卸载 App 后 APNs 会告诉你这个 token invalid，我得有个回收流程把失效 token 删掉，不然会一直往死 token 发。

用户偏好表，数据量固定、字段结构化、要支持“这个用户关没关营销类通知”这种查询，我会放 MySQL 这种关系型库，一个 user_id 对应一行或者对应几行（按通知类别拆）。这块读多写少，前面挂一层 Redis cache 挡住绝大部分读。

模板表，也放 MySQL，字段是 template_id、语言、渠道、标题模板、正文模板。模板改动不频繁，我会整个缓存到 Redis 甚至进程内，渲染的时候把 variables 套进去。

发送记录/日志，量最大写最频繁，我不会放关系型库，会写到一个能扛高吞吐写入的地方——要么是 Cassandra 这种宽列库，要么先进 Kafka 再落到数据仓库。查询模式主要是按 notification_id 或 user_id 查状态，加 TTL 过期。

幂等记录，我会用 Redis 存 idempotency_key，设一个比如 24 小时的过期时间，值就是这条通知的处理结果。判重的时候先查 Redis，命中就直接返回。

高层架构

我把整条链路串一下数据怎么流。

最前面是一个 **Notification API / 接入服务**，上游业务方或者运营后台调它。它做的事很轻：校验参数、查幂等键（如果这个 `idempotency_key` 已经在 Redis 里了就直接返回，不往下走）、然后把这条请求塞进 **消息队列**（我会用 Kafka）就立刻返回 202 给调用方。这一步是解耦和削峰的关键——上游不用等我们真的发出去，尖峰全被 Kafka 缓冲住了，下游按自己的消费能力慢慢拉。

Kafka 后面是 **Notification Worker（消费者）**。Worker 从队列里拉消息，做几件事：查用户偏好（这一类通知用户关了没、现在是不是他的免打扰时段，是的话就丢弃或者延后）、查 device token、用 `template_id` 加 `variables` 渲染出最终文案，然后发给对应的渠道。

这里有个我会特意强调的设计：**Worker 按 provider 分成物理隔离的独立池**——APNs 一个池、FCM 一个池、短信网关一个池，不是一个大池子里按平台分流。这样买到三个独立：**独立扩容**（FCM 忙不用动短信）、**独立故障**（APNs 挂了不拖垮其他渠道）、**独立调优**（推送要求秒级送达，邮件容忍分钟级延迟，两者的超时和重试策略本来就该不一样）。Worker 水平扩，Kafka 的分区数决定并行度上限。

怎么发给第三方，这块值得讲细一点，因为它跟直觉不太一样。APNs 和 FCM 都是 **HTTP/2 长连接**，生产系统维护一个**持久连接池**（几十条常驻连接），靠多路复用往上推，而不是每条通知建一次连接——建连和 TLS 握手的开销比推送本身大得多。所以**限速的实质是控制连接池的并发度**，不是在业务代码里加一个每秒计数器。FCM 的上限是 10 万/秒/project，APNs 没公开硬性数字，靠并发度自己压测调。

失败要先分类，这是最关键的一步：

- **永久失败**（token 无效、用户卸载了 App、payload 超过 APNs 的 4KB 限制）——**绝不重试**，重试多少次都是失败。token 无效的直接标记失效走回收流程，不然会一直往死 token 发。
- **瞬时失败**（限流、5xx、超时）——才进重试，**指数退避而且必须带 jitter（随机抖动）**。这里我会补一句：几千个 worker 同时按固定间隔重试，跟对自己的 provider 发起 DoS 攻击没有区别，jitter 就是为了把重试打散开。

重试 N 次还不行就进**死信队列（DLQ）**人工排查——这条坏消息被停在一边等人看，既不会丢，也不会堵住后面正常的流量。

至于要不要做**分钟级以上的长重试**，我倾向不做——推送有时效性，一条延迟半小时才发出去的通知价值本来就不大。如果换成支付成功、订单状态这类价值不随时间衰减的通知，才需要长重试，那时候我会用 Redis Sorted Set 做延迟队列（score 存执行时间戳）或者换 RabbitMQ 用它的 TTL + 死信交换机，而不是在 Kafka 上凑分档 topic——Kafka 本来就没有延迟消费的能力，硬凑很别扭，而且消息在 Worker 里长时间 sleep 会导致 offset 提交不了、超过 `max.poll.interval.ms` 还会被判定消费者掉线触发 rebalance。

第三方回调回来的送达/点击回执，我再收进来更新发送记录，这块又是一个 webhook 进来落队列的模式。整条线下来，各组件之间全靠 Kafka 解耦，任何一段慢了或者挂了都不会把上游拖死。

关键取舍与瓶颈

第一个核心难点是**去重和幂等**。分布式系统里重试是常态，网络超时了上游会重发，Worker 消费 Kafka 也是 at-least-once（至少一次）语义、同一条消息可能被消费两遍。所以我在两个地方防重复：接入层用 `idempotency_key` 查 Redis 挡住上游的重发；Worker 真正调第三方之前再用一个 `dedup key`（比如 `user_id` 加 `template_id` 加时间窗）查一次，确保同一条通知只发一次。取舍是这会引入一次额外的 Redis 查询开销，但对通知系统来说不重复发是硬指标，值得。

第二个是**削峰和背压**。前面说了峰值能到十几万 QPS，中间这层 Kafka 就是干这个的——上游随便打，下游按 APNs/FCM 能接受的速率慢慢消费。这里的取舍是牺牲了实时性换稳定性，营销 push 晚个几十秒送到完全能接受，但要是没有队列直接怼第三方，一定会被限流甚至打挂。

第三个是**扇出（fan-out）问题**，主要在群发场景。运营发一条营销 push 给一千万人，我不能在一个请求里同步展开一千万条，那会超时。我的做法是接入层只把“campaign + 圈选条件”这一条消息落队列，专门有一批 fan-out worker 去把它展开成一千万条子任务再塞回队列，边展开边消费，这样内存和延迟都可控。

第四个是**第三方的不可靠性**，APNs/FCM 会限流、会超时、token 会失效。前面讲的失败分类（永久 vs 瞬时）、带 jitter 的指数退避、死信队列、失效 token 回收，就是应对这个的一整套。这里一个要提醒面试官的点是：**重试一定要配合幂等**，不然重试本身就会造成重复发送，这两块是绑在一起的——这也是为什么上游那个 idempotency_key 不只是给调用方防重，Worker 侧的重试同样依赖它。

第五个如果要继续扩展，我会讲**优先级隔离**。事务性通知（比如登录验证码、“有人给你转账了”）比营销 push 重要得多，绝对不能因为一波营销群发把验证码堵在队列后面。所以我会给不同优先级的通知走不同的 Kafka topic 或者不同的 Worker 池，高优先级的单独一条快车道，保证它不受群发尖峰影响。这个隔离在真实系统里往往是最后压垮体验的那根稻草，所以我会主动提出来。

题 5 — Chat / IM 实时消息

需求澄清

我一般上来先跟面试官对功能边界。这道题我会问：我们是先做一对一私聊，还是群聊也要？我假设两个都要，但一对一先做，群聊作为扩展。然后要不要支持在线状态（就是那个“对方在线/最后上线时间”的绿点）？要不要已读回执（“已送达”、“已读”）？要不要历史消息拉取、要不要发图片视频这种富媒体？我一般把核心圈定成：发消息、收消息（实时）、拉历史、在线状态、已读、群聊 fanout。富媒体我会说“消息体里只存一个媒体 URL，文件本身走对象存储 + CDN，跟文本消息一个通道”，先不展开。

非功能这块，IM 最关键的三个词我会主动说出来：低延迟（消息端到端最好几百毫秒以内，用户对聊天的延迟极其敏感）、高可用（掉线要能自动重连、消息不能丢）、还有消息的顺序性和不重复。一致性上我会说 IM 通常可以接受最终一致（eventual consistency），但“同一个会话里消息的先后顺序”必须保证，不能你先发的后到。

规模估算

先拍个用户数，假设 DAU（日活）5000 万，平均每人每天发 40 条消息。那每天写入的消息量就是 $5000 \text{ 万} \times 40 = 20 \text{ 亿条}$ （五千万人乘每人四十条，一天二十亿条消息）。

换算成写 QPS：写 $QPS = 20 \text{ 亿} / 86400 \approx 23000$ （也就是一天的消息总数除以一天有 86400 秒，得到每秒大约两万三千次写）。读通常比写高，因为一条群消息要 fanout 给很多人、还有历史拉取，我一般按读是写的 5 到 10 倍估，那读 QPS 大概在 10 万到 20 万这个量级。

再估长连接数。因为是 websocket 长连接，在线用户是要一直挂着连接的，不是请求完就断。假设同时在线是 DAU 的 20%，那并发连接 $= 5000 \text{ 万} \times 20\% = 1000 \text{ 万条长连接}$ （五千万日活的两成同时在线，也就是一千万条常驻连接）。单台机器扛长连接的能力有限，假设一台调优后扛 50 万条，那连接层就需要 $1000 \text{ 万} / 50 \text{ 万} = 20 \text{ 台}$ （一千万连接除以每台五十万，需要二十台接入机）。这个数字我会拿来引出后面“连接层要单独一层、要能水平扩”的设计。

存储上，假设一条消息平均 300 字节（含元数据），一天 20 亿条就是 $20 \text{ 亿} \times 300 \text{ 字节} \approx 600 \text{ GB/天}$ （二十亿条乘每条三百字节，一天大约六百 GB）。一年就是 200 多 TB，这个量级明确告诉我：消息必须存在能水平扩展、能按时间/会话分片的库里，单机 MySQL 肯定放不下。

API 设计

实时收发我会用 websocket，不用 HTTP 轮询。客户端连上来之后，发消息就是往这条 websocket 连接里推一个 send 消息，服务端收消息也是从这条连接推下去。核心几个我会列：连接建立 `connect(userId, token)`（带鉴权 token 建立长连接）；发消息 `sendMessage(conversationId, clientId, content)`，这里我特意带一个 clientId（客户端生成的去重 ID），用来做幂等，网络抖动重发时服务端靠它去重；已读上报 `markRead(conversationId, lastReadMsgId)`；还有历史消息这种非实时的我会走普通 HTTP GET `/conversations/{id}/messages?before=cursor&limit=50`（按游标翻页拉历史，不占长连接）。在线状态可以客户端定期发心跳，也可以服务端根据连接是否存活来推断。

数据模型

我会分几张表/几个库来讲。消息本身 (messages) 字段大概是 message_id、conversation_id、sender_id、content、created_at、还有个序号 seq。存哪很关键：因为写入量巨大、又基本是按 conversation 追加、很少更新，我会选一个宽列/LSM 类的库，比如 Cassandra 或 HBase 这种，用 conversation_id 做 partition key、用 seq 或时间戳做 cluster key（也就是同一个会话的消息物理上排在一起、按顺序存），这样拉某个会话的最近 50 条就是一次高效的范围扫描。我会跟面试官解释为什么不用 MySQL：消息是写多、追加为主、单会话有序，正好是这类库的强项，而且它天生按 partition key 做 sharding，好水平扩。

会话表和用户的收件箱 (inbox / 会话列表) 我会单独放，可以放 MySQL 或者 Redis：记录每个用户有哪些会话、每个会话的最后一条消息、未读数。这个是读密集的（每次打开 App 都要拉会话列表），适合缓存。

在线状态和"用户当前连在哪台接入机上"这种映射我会放 Redis：key 是 userId，value 是他连的那台 gateway 机器 ID，带个 TTL 靠心跳续期。这个数据变化快、要求低延迟查、丢了也能重建，正好用 Redis。

已读回执我用每个会话每个用户的 last_read_seq 来表达（记一个"读到哪个序号了"的水位，而不是给每条消息记一个已读标记），这样存储省很多，判断某条消息有没有读只要比 seq 大小。

高层架构 —— 数据怎么流

我从一条消息的旅程讲。用户 A 发消息，先打到 load balancer，再到接入层 (connection gateway，就是维护 websocket 长连接那一层，前面估的那二十台)。gateway 收到后不直接写库，而是把消息丢给后面的消息服务 (chat service)，chat service 做三件事：分配 seq、写进消息库 (Cassandra 那类)、更新会话的最后消息和未读数。

然后是投递给 B。这里的关键是 A 和 B 大概率连在不同的 gateway 上。所以我会中间放一个 Redis (或专门的 session registry) 记录"B 现在连在哪台 gateway"。chat service 查到 B 在 gateway-7 上，就把消息通过内部通道推给 gateway-7，gateway-7 再从它维护的那条 websocket 把消息推给 B。如果 B 不在线 (Redis 里查不到活跃连接)，那就不推，只落库，等 B 下次上线时通过"拉离线消息 / 拉会话列表未读"补上。

我这里会引入一个消息队列 (Kafka) 来解耦。chat service 写完库之后，把"要投递给谁"这件事丢进 Kafka，由下游的 push/投递服务去消费、去做在线推送或者离线 push notification。这样接入层和投递逻辑解耦，削峰，而且天然能做多端同步（一个人手机和电脑都在线，就往他的多条连接都推）。这块消息 fanout + 队列削峰，我在 Newsbreak 做 feed 和推送相关的东西时用过类似的思路，用 Kafka 把写入和下游多个消费方解耦开。

关键取舍与瓶颈

第一个核心难点是消息顺序。我不会依赖各机器的物理时钟，因为多台机器时钟不完全同步，用墙上时间排序会乱。我会让 chat service 在会话维度给每条消息发一个单调递增的 seq（同一个 conversation 内严格递增），客户端和存储都按这个 seq 排序，这样保证同一会话里"你先发的一定排在前面"。跨会话不需要全局有序，所以不用搞一个全局单点发号器，按会话分开发号就行，扩展性好很多。

第二个是群聊 fanout 的读写放大。一对一好办，一条消息一次投递。群聊如果是"写扩散" (write fanout，发一条就往每个成员的收件箱各写一份)，一个万人大群发一条消息就要写一万次，写会爆炸；如果是"读扩散" (read fanout，只存一份，成员各自来拉)，写很省但大群在线成员同时拉会读爆炸。我的取舍是混合：小群用写扩散（成员少，写得起，读的时候快），大群用读扩散（只存一份，谁在线就实时推给谁的连接，离线的等他自己拉）。这个门槛值可以设个几百人。

第三个是长连接层的水平扩展和有状态问题。gateway 是有状态的（它记着哪些用户连在自己身上），所以不能随便重启、扩缩容也要小心。我的做法是把"用户在哪台 gateway"这个映射外置到 Redis，gateway 本身尽量无状态化到"只是个连接持有者"，某台挂了客户端自动重连到别的机器、重新注册映射即可。load balancer 对长连接要做好会话保持和优雅摘除。

第四个是可靠投递和去重。网络会抖，我会做：客户端发消息带 clientMsgId，服务端靠它幂等去重；服务端投递成功要有 ack，没 ack 到就重推，因为重推可能重复，所以接收端也要靠 message_id 去重。已读和"送达"回执也走同样的通道回传。

第五个如果还有时间我会提在线状态的成本。给每个好友实时精确推在线状态其实很贵（状态一直在变，fanout 很大），我会说这块通常做降级：不用实时精确，改成“最后活跃时间”这种粗粒度，或者只在打开某个会话时按需查一次对方状态，避免把 presence 的 fanout 做成系统瓶颈。

题 6 — 分布式 KV store / 缓存

需求澄清

拿到这道题我不会急着画架构，先跟面试官对齐我们到底在做什么。分布式 KV store 和缓存其实是两个相关但不完全一样的东西：一个是持久化的存储（数据要一直在，类似 DynamoDB、Redis 当主存那种用法），一个是挡在数据库前面的 cache（数据丢了没关系，回源再拉一次）。我会先问一句，这题是想让我做一个能当主存用的、数据不能丢的 KV store，还是做一个给下游数据库挡流量的缓存层？这两个假设会直接影响我后面要不要强调持久化和副本一致性。我下面默认往“分布式 KV store，但同时要能当缓存用”这个方向讲，因为它覆盖面最广。

功能需求上其实很简单，就三个核心操作：put(key, value)（写入一个键值对）、get(key)（按 key 读出 value）、delete(key)（删掉一个 key）。可能再加一个可选的 TTL，就是这个 key 存活多久之后自动过期，这个对缓存场景特别重要。value 我会先假设是不透明的 bytes，不做结构化查询、不支持按 value 搜索、也不支持范围扫描——KV store 的定位就是按 key 精确查，不是数据库。

非功能需求这块我觉得才是这题真正考的东西。我会跟面试官强调几点：第一是低延迟，读写最好都在个位数毫秒；第二是高可用，某台机器挂了整个系统还得能服务；第三是可水平扩展，数据量和流量涨了我加机器就行，不用重构；第四是这里有个经典取舍，就是一致性和可用性我更偏向哪边。我会说 KV store 这类系统通常选 AP（也就是优先保证可用性和分区容忍，一致性上放宽到最终一致），因为大多数用它的场景（session、缓存、计数器）都能接受读到稍微旧一点的数据，但不能接受服务不可用。

规模估算

我会先跟面试官拍几个数字，方便后面判断要多少机器。假设我们有 1 亿日活用户，每个用户平均每天产生 100 次 KV 操作，那每天总请求量就是 100 亿次。换成 QPS 我会这么算： $QPS = \text{每天请求量} / 86400$ （也就是一天的总请求数，除以一天有多少秒，86400 就是 24 乘以 3600）。100 亿除以 86400 大概是 11.5 万 QPS 的平均值。但真实流量不是平的，白天高峰会翻好几倍，所以我一般会把峰值估成平均的 3 到 5 倍，也就是峰值 $QPS \approx \text{平均 QPS} \times 5$ （把平均值乘以 5 当作峰值兜底），大概到 50 到 60 万 QPS。这个量级已经不是一台机器能扛的了，天然就需要 sharding。

读写比例我会假设是读多写少，大概 read : write = 10 : 1（每来 10 次读大概才有 1 次写），这是缓存类系统的典型特征，也决定了我后面会重点优化读路径、加副本来分摊读。

存储估算我这么拍：假设我们要存 10 亿个 key，每个 key 加 value 平均 1KB，那原始数据量 = $10 \text{ 亿} \times 1\text{KB} = 1\text{TB}$ （十亿个键值对，每个一千字节，乘起来就是 1TB）。但这只是一份数据，为了高可用我要做副本，一般存 3 份，所以实际占用 = $1\text{TB} \times 3 = 3\text{TB}$ （原始数据乘以副本数 3）。3TB 单机放不下也不安全，这又一次说明必须 sharding，把数据切开摊到很多台机器上。这两个估算结论其实是一致的：不管从 QPS 还是从存储看，单机都不够，必须分布式。

API 设计

对外我会设计得极简，就是围绕那三个操作。put 我会写成类似 PUT /keys/{key}，body 里放 value，可选带一个 ttl 参数表示过期时间；get 就是 GET /keys/{key}，返回 value，如果不存在就返回 404；delete 就是 DELETE /keys/{key}。这里我会说实际生产里 KV store 通常不用 HTTP，而是用更省的二进制协议（像 Redis 的 RESP 协议），因为 HTTP header 的开销对这种每次就传几十字节 value 的场景太重了，但面试里我先用 REST 表达清楚语义就行。

有个细节我会主动提一下：put 我倾向设计成幂等的，也就是同一个 key 重复写同样的值，结果应该一样，这样客户端超时重试的时候不会出问题。如果要支持并发控制，我可以再加一个版本号（比如 compare-and-set，带上你读到的版本号才允许写），但那是进阶，entry-level 我先不展开。

数据模型

数据模型本身很直白，就是 key 到 value 的映射，加上一点元数据比如 TTL 和版本号。真正要讲的是"存在哪、用什么"。

在单台机器内部，我需要一个能快速按 key 查的结构。如果数据全放内存（缓存场景），我就用一个 hash table，get/put 都是接近 $O(1)$ （也就是查一个 key 基本是常数时间，不随数据量增大而变慢）。如果要持久化、数据量超过内存，我会用一个基于 LSM-tree 的存储引擎，比如 RocksDB，它对写特别友好——写先进内存和 append-only 的 log，之后再批量刷到磁盘，这样写吞吐很高，这也是很多 KV store（Cassandra、DynamoDB 底层）选它的原因。我会跟面试官说清楚这个选择：读多写少但写要快、value 无结构、不需要 join 和范围查询，这种形态用关系型 MySQL 是浪费的，KV 引擎更合适。

为什么不用 MySQL？我会一句话带过：关系型数据库擅长事务、join、复杂查询，但我们这里全是按主键的点查，用不上这些能力，反而扛不住这个 QPS，所以选专门的 KV 引擎。

高层架构

我先讲数据怎么流。客户端的请求先打到一个 load balancer 或者一层路由层，这层负责把请求转发到正确的机器上。核心问题是：这么多台机器，一个 key 该去哪台？这就是 sharding 要解决的。

最朴素的做法是取模， $\text{shard} = \text{hash}(\text{key}) \% N$ （把 key 哈希之后对机器数 N 取余数，得到落在第几台）。但这个做法有个致命问题：一旦机器数 N 变了（加机器或者挂了一台），几乎所有 key 的取模结果都变了，等于要把全部数据重新搬一遍，这在生产里是灾难。所以我会引入 consistent hashing（一致性哈希）：把哈希空间想象成一个环，每台机器和每个 key 都哈希到环上的某个点，一个 key 归属它顺时针遇到的第一台机器。这样加一台或挂一台，只影响环上相邻的一小段 key，大部分数据不用动。我还会补一句实际会用 virtual node（虚拟节点），就是让每台物理机在环上占很多个点，这样数据分布更均匀，也不会因为某台机器哈希位置不好就热点倾斜。（如果被追问真实经验：可以提 MongoDB 分片集群的形态——路由层无状态不存数据、数据按 shard key 切在多个 shard 上、每个 shard 内部还是主从副本集，所以 sharding 解决容量和写入、replication 解决故障和读扩展，两件事是正交的。分片键选错了后面很难改，跨分片 scatter 查询没索引会很慢。别主动展开到具体产品的内部实现。）

分好片之后，每个 shard 我不会只放一台机器，而是放一组副本，通常一主多从或者对等的 3 副本。写的时候写主副本，然后异步或半同步复制到其它副本；读的时候可以从任意副本读，这样读能被多台分摊，正好配合我们读多写少的特点。这里就落到那个一致性取舍上了：如果我允许从副本读，就可能读到还没同步过来的旧值，也就是最终一致（数据最终会一致，但某一刻可能不一致）；如果我要强一致，就得读主或者做多数派确认，延迟和可用性都要付代价。我会明确说这题我选最终一致，因为可用性和延迟对 KV/缓存更重要。

如果这个系统是当缓存用（挡在数据库前面），那读路径是这样的：先查 cache，命中就直接返回；没命中就回源去数据库拉，拉到之后写回 cache 再返回，这叫 cache-aside（也就是缓存旁路，应用自己负责在没命中时回填缓存）。写路径通常是先写数据库，再把 cache 里对应的 key 删掉，让下次读重新回源，这样避免 cache 和数据库长期不一致。

关键取舍与瓶颈

第一个是 sharding 带来的扩容代价，我上面讲的 consistent hashing 就是为了让加减机器时只搬一小部分数据。但即便如此，扩容时的数据迁移还是要小心做，边搬边服务，不能一次全停。这是这题最核心的分布式难点。

第二个是热点 key 问题。consistent hashing 保证的是 key 大致均匀，但挡不住某一个 key 特别热（比如某条爆款内容的 id）。这种单点热 key 会把某台机器打爆。我的办法是对热 key 做额外副本、或者在客户端本地再加一层小缓存挡一部分，甚至把这个 key 拆成多个带后缀的子 key 分散开。

第三个是缓存三兄弟，这题基本必问。缓存穿透，是指大量请求查一个数据库里根本不存在的 key，每次都命中不了 cache、直接打到数据库；我的解法是把"查不到"这个结果也缓存下来（缓存一个空值并给短 TTL），或者用 bloom filter（一个能快速判断"这个 key 肯定不存在"的概率结构）在前面挡掉。缓存击穿，是指某个热 key 刚好过期的一瞬间，大量并发同时没命中一起去回源，把数据库压垮；我会用一把互斥锁让只有一个请求去回源、其它等着，或者干脆给热 key 设成不过期后台定时刷新。缓存雪崩，是指大量 key 在同一时刻集中过期、或者 cache 集群整体挂

掉，导致流量瞬间全压到数据库；解法是给 TTL 加一个随机抖动让过期时间错开（比如基础 TTL 上下浮动几分钟），以及给 cache 集群本身做高可用。

第四个是内存有限的淘汰策略，我会用 LRU（least recently used，最近最少使用），也就是内存满了就把最久没被访问过的 key 踢掉。实现上通常用一个 hash table 加一个双向链表：hash table 保证 $O(1)$ 找到 key，双向链表维护访问顺序，每次访问把节点挪到头部，淘汰就从尾部删（查找常数时间、更新顺序也是常数时间）。如果访问模式里有些低频但会突然扫一遍的场景，LRU 会被冲掉，那可以考虑 LFU（按访问频率淘汰）之类的变种，但 entry-level 我 LRU 讲清楚就够。

第五个如果面试官想再往下推，我会提复制带来的一致性和故障处理：主副本挂了怎么选新主、异步复制丢没丢数据、跨机房部署时延迟怎么办。这些我会点到为止，说明我知道有这些坑，真正深入可以展开写复制协议，但核心思路就是在可用性和一致性之间按业务需求做取舍。

题 7 — Web Crawler 爬虫

需求澄清

拿到这道题我不会马上开始画框图，先跟面试官对齐范围。功能上我会先问：这个 crawler 是要爬整个 web（像搜索引擎那种通用爬虫），还是只爬某几个特定站点？要不要抽取内容、建索引，还是只负责把页面抓下来存起来？只爬 HTML 文本，还是图片、PDF、视频也要？要不要支持增量重爬（同一个页面隔一段时间再抓一次看有没有更新）？我一般会吧范围定在：给一批 seed URL（种子 URL），从这些页面出发不断顺着链接往外爬，把 HTML 抓下来存储，同时把新发现的链接放回队列，支持后续重爬。

非功能方面我关心几点：一是规模，要爬多少页、多长时间爬完，这决定了整体吞吐；二是礼貌性（politeness），不能把人家站点打挂，得遵守 robots.txt，同一个域名不能猛打；三是可扩展性，得能水平扩展成分布式；四是容错，机器挂了任务不能丢；五是去重，同一个 URL 不能反复爬，同一份内容也尽量别重复存。这块的取舍我在 Newsbreak 做内容抓取相关的东西时体会挺深的，礼貌抓取和吞吐经常是打架的。

规模估算

假设面试官说要在一个月內爬 10 亿个页面。先算 QPS（每秒抓多少个页面）： $QPS = 10\text{亿} / (30 \times 86400)$ （也就是十亿个页面除以三十天、每天八万六千四百秒），算出来大概是 386，我一般直接说约 400 QPS。峰值我会乘个 2 到 3 倍，按 1000 QPS 左右来设计。

再算存储。假设每个页面平均 500 KB 的 HTML：总存储 = $10\text{亿} \times 500\text{KB}$ （也就是十亿乘以每页五百千字节），等于 500 TB。这还只是原始 HTML，如果要存解析后的结构、还要留历史版本重爬，我会往上翻几倍，按 PB 级别来预估，所以存储肯定得上对象存储而不是塞数据库。

带宽也顺手算一下： $400\text{ QPS} \times 500\text{KB}$ （也就是每秒四百个页面、每个五百千字节），大概是 200 MB/s 的下行带宽，换算过来接近 1.6 Gbps，这告诉我单机肯定扛不住，必须分布式多机去抓。

API 设计

这题严格说不是一个对外的用户 API，更多是系统内部的接口，但我会讲几个关键的内部契约。对外可以暴露一个提交种子的接口，比如 `submitSeeds(urls[])`（把一批种子 URL 丢进系统），返回一个 `jobId`。另一个是查询进度 `getCrawlStatus(jobId)`（查这个爬取任务爬了多少、还剩多少）。

系统内部真正核心的是两个操作：一个是 `fetcher` 从 URL frontier 里 `dequeue` 一个 URL（取出下一个该抓的链接），另一个是把新解析出来的链接 `enqueue` 回 frontier（塞回队列等着被抓）。这两个操作围绕着 URL frontier 这个队列转，是整个系统的核心。

数据模型

我会分几块存。第一块是 URL frontier 本身，也就是待抓队列，这个我会用 Kafka 或者 Redis 这类支持高吞吐入队出队的东西来做，里面存的就是 URL 加上一点元数据，比如优先级、发现时间、所属域名。为什么不用 MySQL？因

为它是高频读写的队列语义，用消息队列或 Redis 更合适。

第二块是去重用的结构。已经见过的 URL 我用一个 bloom filter（布隆过滤器）来判重，它内存占用极小、查得飞快，代价是有小概率误判（说见过其实没见过），但对爬虫来说漏爬极少量页面是可以接受的。这就是一个基本取舍：用一点点准确性换巨大的内存节省。10 亿个 URL 如果用普通 hash set 存会占几十上百 GB，bloom filter 可能几个 GB 就够了。

第三块是抓回来的原始页面，这个我存对象存储，比如 S3 这种，因为是 PB 级的大 blob，按 key（比如 URL 的 hash）存取就行，便宜、能无限扩。

第四块是元数据库，存每个 URL 的抓取状态、最后抓取时间、HTTP 状态码、内容的 hash 值等等，这块查询模式比较结构化、量也大，我会用一个可以 sharding（分片）的数据库，比如 Cassandra 或者分库分表的 MySQL。内容 hash 存在这里是为了做内容级去重——两个不同 URL 抓回来内容一模一样（比如带 tracking 参数的镜像页），我通过比对内容 hash 就能发现，不重复存。

高层架构

数据流我一步步说。一开始种子 URL 被塞进 URL frontier。fetcher（抓取器）是一堆 worker，它们不停地从 frontier 取 URL 出来。取到之后第一件事不是直接抓，而是先查这个域名的 robots.txt——我会有一个专门的模块负责下载和缓存各站点的 robots 规则，判断这个 URL 允不允许爬、爬取间隔是多少。这就是礼貌抓取（politeness）的核心。

通过了 robots 检查，fetcher 通过 DNS 解析拿到 IP，然后发 HTTP 请求把页面抓下来。DNS 这一步我会单独提一句，因为 DNS 解析经常是瓶颈，我会加一层 DNS cache（DNS 缓存），不然每次抓都去解析域名会很慢。

抓回来的 HTML 先扔进对象存储存原始副本，同时交给 parser（解析器）。parser 干两件事：一是从页面里抽出所有的链接（outlinks），二是算这个页面内容的 hash 做内容去重。抽出来的每一个新链接，先过 URL 归一化（把 http/https、末尾斜杠、大小写、去掉无关参数这些统一一下，不然同一个页面会有一堆变体 URL），然后拿去问 bloom filter：见过没有？没见过就 enqueue 回 URL frontier，见过就丢弃。这样系统就形成了一个闭环，源源不断地自己给自己喂新 URL。

整个系统是分布式的：fetcher、parser 都是无状态的 worker，可以水平加机器；frontier 用消息队列天然支持多消费者；去重和元数据都是可以 sharding 的存储。这样任何一台机器挂了，任务还在队列里，别的 worker 接着干，容错就有了。

关键取舍与瓶颈

第一个也是这题最核心的难点，是礼貌抓取和吞吐的矛盾。我既要 1000 QPS 的总吞吐，又不能对单个域名猛打。解法是 URL frontier 不能是一个大队列平铺，而要按域名分成很多子队列，每个域名一个逻辑队列，然后用一个调度层保证同一个域名两次抓取之间有间隔（比如按 robots 里的 crawl-delay），但不同域名之间可以高度并行。这样整体吞吐很高、单站点又很温柔。这块的设计我觉得是这道题真正想考的东西。

第二个是去重的取舍。这里我会先说清楚 bloom filter 的误判方向是结构决定的、反不过来：插入时把几个哈希位都置 1，位只增不减，所以查询时只要有一位是 0，就能确定这个 URL 没见过；全是 1 却不能确定，可能是别的 URL 凑巧覆盖了这几位。也就是说假阳性（说见过其实没见过）不可避免，假阴性不会发生——这个不对称改不了，除非换成 Counting Bloom Filter 或 Cuckoo Filter，但那两个引入假阴性是为了支持删除，不是为了调转误判方向。

放到爬虫场景，假阳性就是漏爬：误判为见过就直接跳过，这个页面再也爬不到了。所以方案有三个层次：

1. **纯 bloom filter**——最省内存最快，代价是极少量漏爬。大多数场景够用，爬全网本来也不追求一个不漏。
2. **纯数据库精确判重**——不漏但慢且贵，每个 URL 都要一次网络查询，在每秒几万 URL 的吞吐下是实打实的成本。
3. **两层叠加（我的首选）**——bloom filter 说“没见过”就直接爬，因为这个否定是确定的；只有它说“见过”时才去数据库精确确认一次。这样 99% 以上的查询在内存里就解决了，只有一小部分付出数据库查询的代

价，既不漏爬又保住了性能。

所以这不是“用 filter 还是用数据库”的二选一——bloom filter 本来就不该当最终裁决者，它是个便宜的前置筛。只有当内存存在装不下 filter、或者漏爬完全无所谓时，才会退化成方案 1 或 2。

第三个是 crawler trap（爬虫陷阱）和无限空间。有些站点会动态生成无限多的 URL，比如日历页面可以一直点下一个月，或者故意做死循环链接。我会加限制：每个域名最多爬多少页、URL 最大深度多少、单页抽出的链接数设上限，防止被一个站点拖死。

第四个是扩展和热点问题。当页面量涨到几百亿，frontier 队列和去重结构会成为瓶颈，我会按域名的 hash 做 sharding 把负载摊开。热点是某些超大站点（比如新闻门户）会贡献海量链接，容易让某个 shard 过载，这时候要么给大站点单独分配更多抓取配额和独立队列，要么在调度上做限流。

第五个是增量重爬。页面会更新，我不能爬一次就完事。我会给每个 URL 记一个重爬周期，热门/更新频繁的页面爬得勤，静态页面爬得稀，靠元数据库里的最后抓取时间加一个优先级策略来调度重新入队。如果要再往上扩，我会引入基于历史变化频率的预测，让重爬调度更聪明。

题 8 — Typeahead 搜索自动补全

需求澄清

我上来会先跟面试官对齐一下我们到底在做什么。Typeahead 就是用户在搜索框里每敲一个字符，系统就实时返回一串补全建议，比如敲 "new" 就弹出 "newsbreak"、"news today"、"new york weather" 这些。所以功能需求上，我关心几点：第一，用户输入一个前缀，我要返回 top-k 个建议，k 一般就是 5 或者 10 个；第二，这些建议要按热度排序，越多人搜的排越前面；第三，得支持随着输入不断刷新，用户每多敲一个字母就重新请求一次。

然后我会问几个边界问题：要不要支持中间词匹配（比如输入 "york" 能不能匹配到 "new york"），还是只匹配前缀？entry-level 我先假设只做前缀匹配，这样最简单，后面有时间再聊扩展。要不要做拼写纠错、个性化（根据用户历史推荐）？这些我会先标记成 nice-to-have，先把主干做出来。

非功能需求这块是这题的灵魂，我会强调：延迟必须极低，因为用户每敲一个键就要一次响应，体感上超过 100 毫秒就觉得卡了，所以我心里的目标是端到端 50 到 100 毫秒；其次是高可用和高并发，搜索是入口流量很大；最后是数据可以稍微有点延迟——就是说今天新冒出来的热词，晚几个小时甚至一天才进补全库，完全可以接受。这个“允许陈旧”的点很关键，它直接决定了我后面可以走离线更新的路线。

规模估算

先估 QPS。假设我们有 1000 万日活用户（DAU = 10M），每人每天做 10 次搜索，每次搜索平均敲 5 个字符，而每敲一个字符都要请求一次补全。那么每天的补全请求量大概是 $10M \times 10 \times 5 = 5$ 亿次（也就是日活乘以人均搜索次数再乘以每次搜索的击键数，等于五亿次请求每天）。

换算成 QPS： $QPS = 5\text{亿} / 86400 \approx 5800$ （也就是每天的总请求数除以一天有多少秒，约等于每秒五千八百次请求）。这是平均值，我会顺手把峰值也估一下，一般峰值取平均的两到三倍，所以峰值 $QPS \approx 5800 \times 3 \approx 17000$ （平均 QPS 乘以三倍系数，大约每秒一万七千次）。这个量级告诉我，读请求非常密集，读写严重不对称——读是海量的，写（更新热度）其实很稀。

再估存储。假设我们要索引的独立搜索词有 1 亿条（100M 个 unique query），每条平均 20 个字节的文本再加上一些热度、指针之类的元数据，算它 50 字节。那么原始数据量大概是 $100M \times 50B = 5GB$ （一亿条乘以每条五十字节，等于五个 GB）。但注意，Trie 这种前缀树因为共享前缀，实际内存开销会比裸文本大一些（指针多），我一般会乘个 3 到 5 倍的系数，所以估个 15 到 25GB 内存。这个规模一台大内存机器其实塞得下，但为了高可用和抗并发我还是会做分片和多副本。

API 设计

对外就一个核心接口，我会设计成一个简单的 GET：`GET /suggestions?prefix=new&limit=10`（传一个前缀 prefix 和想要的条数 limit，返回补全列表）。返回体就是一个按热度排好序的字符串数组，比如

```
{"suggestions": ["newsbreak", "news today", "new york weather"]}
```

。我会强调这个接口必须是无状态的、可以水平扩展的，而且响应要能被 cache。

后台还有一个更新热度的数据通路，但它不是同步 API，而是走日志收集：每次用户真正提交了一次搜索，前端打一条埋点日志到 Kafka，后台离线消费这些日志去统计热度。所以写路径和读路径是完全分开的，这点我会特意点出来。

数据模型

核心数据结构我选 Trie（前缀树）。为什么用 Trie 而不是数据库的 LIKE 'new%' 查询？因为前缀匹配正是 Trie 的天生强项——沿着字符一路往下走就能定位到某个前缀节点，时间只跟前缀长度有关，跟总词量无关。这里有个关键优化：如果每次查询都从命中的节点往下遍历整棵子树再排序取 top-k，那太慢了。所以我会每个 Trie 节点上预先缓存好“以这个前缀开头的 top-k 建议”（在节点里直接存一个长度为 k 的排好序的列表）。这样查询就变成：走到前缀节点、直接把它存的那个 top-k 列表拿出来返回，几乎是常数时间。

Trie 本身放在内存里，因为要极致低延迟，落磁盘或者走数据库都太慢。持久化方面，我会把这棵 Trie 定期序列化快照存到磁盘或对象存储（比如 S3），服务重启的时候从快照加载，不用从头重建。原始的搜索日志和热度统计结果，我会存在一个便宜的地方，比如 MongoDB 或者数仓表，用来给离线任务算热度。

高层架构

我把整个系统拆成读路径和写路径两条线。

读路径：用户敲字符 → 请求先打到 load balancer → 转发到一组无状态的 suggestion service 实例 → 服务先查 cache（比如 Redis 或者本地内存 cache），命中就直接返回；没命中就去查内存里的 Trie，拿到 top-k，回填 cache 再返回。前缀 cache 特别有用，因为热门前缀（像 "n"、"ne"、"new" 这种短前缀）会被反复查，命中率非常高，能挡掉大部分流量。这条路我要做到毫秒级。

写路径（离线更新热度）：用户提交搜索 → 埋点日志进 Kafka → 一个离线批处理任务（比如每小时或每天跑一次的 Spark/MapReduce 作业）去聚合统计每个 query 的搜索次数 → 算出新的热度、重新计算每个 Trie 节点的 top-k → 生成一份新的 Trie 快照 → 把这份新 Trie 灰度推送/热加载到各个 suggestion service 实例上。这条路可以慢，因为我们前面澄清过数据允许陈旧。这套模式很通用：大流量的行为日志走 Kafka 落下来，离线批算好再回推到在线服务，在线只读预计算结果。

关键取舍与瓶颈

第一个核心点是读写分离 + 允许陈旧。我把“更新热度”这个写操作从在线请求路径里彻底拿掉，改成离线批处理再热加载。取舍是补全结果不是实时的，新热词有延迟；换来的是读路径极其简单极其快，这对一个每秒上万请求、要求 100 毫秒内响应的系统是绝对值得的。

第二个是分布式 Trie（sharding）。当词库大到一台机器内存装不下，或者单机扛不住 QPS 时，我要把 Trie 拆开。最直观的拆法是按前缀首字母或前几个字符分片，比如 a 开头的一台、b 开头的一台。取舍是会有热点问题——某些字母开头的词特别多、特别热（分布不均），需要更细的拆分或者用 consistent hashing 之类的手段让分片更均匀，同时加一个路由层根据前缀把请求发到对的分片。entry-level 我知道“有这个问题、大概按前缀分、会有热点、需要路由”就够了。

第三个是前缀 cache 的层次。我会在多层做 cache：客户端可以缓存已经敲过的前缀结果，CDN/边缘可以缓存热门前缀，服务端再有 Redis。取舍是 cache 越激进越快，但热度更新后的失效就越麻烦——我一般用短 TTL（比如几分钟到一小时过期）来兜底，反正数据本来就允许陈旧，TTL 到了自然刷新。

第四个是延迟优化的工程细节。每敲一个键都请求会打爆后端，所以前端我会做 debounce（用户停顿一小会儿才发请求，比如停 100 毫秒），减少无效请求；返回结果也要控制体积，只回 top-k 不回多余字段。

如果要继续扩展，我会往这几个方向聊：加个性化（结合用户历史/地理位置对 top-k 重排）、加拼写纠错和模糊匹配、支持中间词匹配（这时候纯前缀 Trie 就不够了，可能得引入 n-gram 或者倒排索引）。但主干我会守住：内存

Trie + 节点预存 top-k + 前缀 cache + 离线更新热度，这套组合足以支撑一个低延迟、高并发的 typeahead 系统。

题 9 — Top-K / Trending 热榜

需求澄清

我一般先跟面试官把题目框住，因为「热榜」这个词能指的东西太多了。我会先问功能层面的：我们要出的是什麼榜？是全站热搜词的 Top-10，还是每个类目/每个地区各出一份榜？榜单的粒度是什麼——是「过去 1 小时最热」「过去 24 小时最热」还是「实时此刻最热」？这个特别关键，因为它直接决定我要不要做时间窗口。然后我会确认 K 大概多大，是 Top-10 这种小 K，还是 Top-1000 这种大 K；还有输入是什麼，是一串一串的事件流（比如用户点了某个 item、搜了某个词），我要基于这些事件去统计出现次数，然后排出频次最高的那几个。

非功能层面我关心几点：第一是量级，写入（事件）通常远大于读取（看榜），这是这类系统的典型特征；第二是延迟，看榜是用户直接感知的，我希望榜单读取是毫秒级的，最好直接从 cache 里拿；第三是新鲜度，榜单允许有多大延迟，是能接受几秒到几十秒的延迟、还是要求严格实时；第四也是这题的核心——我会主动跟面试官挑明**精确 vs 近似**这个取舍：如果我们能接受榜单有一点点误差（比如第 8 名和第 9 名偶尔排错，但 Top-3 一定对），那我就能用近似算法把成本砍下来一大截。绝大多数「热榜」场景，用户根本不在乎精确计数，所以我会默认走近似方案。

规模估算

先假设我们是中等偏大的产品，每天有 1 亿个事件进来（点击、搜索这类）。那写入的 $QPS = 1\text{亿} / 86400 \approx 1157$ （也就是一天的事件总数除以一天有多少秒，大约每秒一千多次写）。但热榜有个特点是会突发，热点事件一来流量能翻好几倍，所以我会按峰值估，比如乘以 5，峰值写入大概 $QPS \approx 5000$ （也就是平时的五倍左右）。读取这边，假设有一千万日活、每人每天看 10 次榜，那读 $QPS = 1000\text{万} \times 10 / 86400 \approx 1157$ （日活乘以人均次数再除以一天的秒数），量级跟写差不多，但读我完全可以用 cache 扛，不碰底层。

存储上，如果走**精确**方案，我得给每个不同的 item 存一个计数。假设一天有 1000 万个不同的词/item，每个存 key 加计数大概 100 字节，那一天就是 $1000\text{万} \times 100\text{字节} = 1\text{GB}$ （不同 item 数乘以每条的字节数，大约一天一个 G），这个量其实完全存得下。但如果 item 空间特别大——比如 URL、query 这种能有上亿种、甚至无限种，精确存就开始爆内存了。这时候**近似**方案的价值就出来了：count-min sketch 用一个固定大小的二维数组就能估算任意多 item 的频次，比如开一个 5 行 $\times$ 27 万列的 int 数组，大概几 MB 到十几 MB，内存是**恒定的**，不随 item 种类增长。这就是我前面说要不要接受近似的那个取舍的具体收益。

API 设计

对外其实就两个接口。写入这边我一般不直接暴露一个「加一」的 HTTP 接口，而是让事件走 Kafka：业务方把事件打到 Kafka 的 topic 里，格式大概是 `recordEvent(itemId, timestamp, count=1)`（记录某个 item 在某个时间点出现了一次）。用 Kafka 而不是同步调用，是因为写入量大又有突发，我需要一个缓冲区把峰值削平，也能解耦上下游。

读取这边就是 `getTopK(category, window, k)`（给定类目、时间窗口和 K，返回排名前 K 的 item 列表和它们的估算频次）。返回大概是一个数组，每项是 item 加上它的分数，按分数从高到低排好序。这个接口直接读 cache，追求毫秒返回。

数据模型 / 存哪、用什么库

我会分三块存。第一块是**近似计数结构**，也就是 count-min sketch，这东西本身就是个内存里的二维数组，放在流处理的计算节点内存里，或者持久化一份到 Redis 也行（Redis 有支持这类概率结构的模块）。它负责回答「这个 item 大概出现了多少次」，好处是省内存、更新快，代价是可能高估（永远不会低估，因为多个 item 会哈希到同一格子累加，所以只会偏大）。

第二块是**维护 Top-K 的 heap（最小堆）**。sketch 只能告诉我某个具体 item 的频次，但它不会自己吐出「谁是前 K 名」。所以我在旁边配一个大小为 K 的 min-heap（堆顶是当前第 K 名，也就是这批里最小的那个）：每来一个 item，我先用 sketch 查一下它现在的估算频次，然后拿这个频次跟堆顶比——如果比堆顶大，就把它换进堆里。用

最小堆是因为我要频繁地「踢掉当前最弱的一个」，堆顶正好就是最弱的，替换是 $O(\log K)$ 的（也就是每次操作的代价是 K 的对数级别， $K=10$ 的时候基本可以忽略）。

第三块是**结果 cache**，我把算好的榜单直接写到 Redis 里，key 大概是 `topk:{category}:{window}`，value 就是排好序的列表。看榜的读请求全部命中这里，跟计算路径完全隔离开。真正的原始事件如果还想留一份做离线精确校对，我会另外落到数据仓库（比如 HDFS/数仓），但那不在实时链路上。

高层架构 / 数据怎么流

我把整条链路串一下。事件先从各个业务端打进 **Kafka**，Kafka 按 item 做 partition（用 itemId 做 key 做一致性哈希，保证同一个 item 永远落到同一个分区、同一个消费者），这样每个消费者只负责自己那一撮 item，各自维护自己的局部 sketch 和局部 top-k，互不干扰、天然水平扩展。

接下来是**流处理层**（我会用 Flink 或 Spark Streaming 这类，或者自己写消费者也行），它做两件事：一是把事件累加进 count-min sketch，二是维护那个 K 大小的 heap。这里我会引入**时间窗口**来实现「过去 1 小时热榜」这种需求——我不是维护一个从开天辟地累加到现在的总数，而是用一串小的时间桶（比如每 1 分钟一个 sketch），要算「过去 60 分钟」就把最近 60 个桶的 sketch 加起来。这样窗口能往前滑动，老的桶到期就丢掉，热点降温后自然会掉出榜单，不会永远霸榜。这就是为什么要做时间窗口而不是简单累加。

因为事件是按 item 分区、分散在多个消费者手里的，每个消费者算出来的只是**局部 Top-K**，所以最后还有一个**汇总（merge）步骤**：一个协调节点周期性地（比如每几秒）把各个分区上报的局部 Top-K 候选收集起来，合并、重新排序，得出**全局 Top-K**，再写进 Redis cache。这里有个小技巧——各分区上报时要多报一点（比如报 Top-2K 而不是刚好 Top-K），因为一个全局排第 5 的 item，在某个分区里可能只排到第 12，报太少会漏掉它。

读路径就很简单了：用户请求 `getTopK`，load balancer 分到某台读服务，直接从 Redis 拿预先算好的榜单返回，毫秒级。写路径（重、突发、可容忍延迟）和读路径（轻、要快、要稳）就这么彻底分开了。

关键取舍与瓶颈

第一个也是最核心的，就是**精确 vs 近似**。精确方案要给每个 item 存真实计数，item 种类一多就爆内存、也难水平扩展；近似方案用 count-min sketch，内存恒定、更新极快，代价是频次可能被高估（哈希冲突导致），偶尔会让排名靠后的名次抖动。我的判断是：热榜这种场景 Top-3、Top-10 的相对顺序对就行，用户根本感知不到第 47 名到底是不是真的第 47，所以近似几乎是划算的。如果哪天真需要精确（比如涉及计费、发奖），我会退回精确计数，或者用 sketch 先做粗筛、再对候选集做一次精确复核。

第二个是**热点 key 打爆单分区**。我按 item 哈希分区，但如果某个词突然爆红（世界杯进球那种），它对应的那个分区就会被打爆，成为热点。我会做的是对超热 key 再拆一层——加个随机后缀把它散到多个分区，最后汇总时再合起来；或者在消费者前面加一层本地预聚合，先在客户端把同一个 item 攒一批再上报，减轻下游压力。

第三个是**时间窗口的粒度取舍**。桶切得越细（比如每秒一个），榜单越实时、滑动越平滑，但要维护的 sketch 份数越多、内存和合并成本越高；切得越粗（每小时一个），成本低但榜单更新一跳一跳的、不够实时。我会根据新鲜度要求选一个折中，比如 1 分钟一个桶。

第四个是**新鲜度 vs 成本的取舍**，也就是 merge 和刷 cache 的频率。刷得越勤榜单越新，但协调节点收集、合并、写 Redis 的开销越大。既然大多数热榜没必要秒级精准，我会把刷新周期设成几秒到几十秒，用一点点延迟换系统的轻松。

第五个是**扩展性和容错**。要扩容，写这边加 Kafka 分区和消费者就行，读这边加 Redis 副本和读服务，两边独立扩、互不影响，这也是我一开始就坚持读写分离的原因。容错上，Kafka 本身持久化、能重放，万一某个流处理节点挂了，重启后从 Kafka 的 offset 接着消费就能恢复；sketch 这种内存结构我会定期做 checkpoint（快照落盘），避免挂一次就全部从头重算。最坏情况丢一点点计数，对近似榜单来说也在可接受范围内。

题 10 — 视频存储与播放（类 YouTube）

需求澄清

一上来我不会急着画架构，先跟面试官对齐这个系统到底要做什么，因为「视频平台」这个词太大了，YouTube、TikTok、Netflix 差别很大。我会先问功能需求：核心就三件事，用户能上传视频、视频上传后能被处理成可以播放的格式、其他用户能流畅地看。至于评论、点赞、订阅、推荐这些我会说先放一边，作为可选项，如果时间够再聊，先把「上传-播放」这条主线撑起来。我还会确认一下要不要支持不同清晰度的自适应播放（也就是网络差的时候自动降到 480p，网络好的时候上 1080p），这个基本是必须的，因为它直接影响到转码这块的设计。

然后是非功能需求，这里我会强调几点。第一，这是一个典型的读多写少系统——上传视频的人远少于看视频的人，一个热门视频可能被播放几百万次但只上传了一次，这个「读多写少」的特性会贯穿我后面所有的设计取舍。第二，播放要低延迟、要能全球分发，所以 CDN 是绕不开的。第三，视频文件很大，存储和带宽是主要成本，可用性要求高（视频不能丢），但一致性可以放宽——一个视频上传完晚几秒才能被搜到、播放量计数晚点更新都无所谓，这是最终一致就够的场景。这块其实我在 Newsbreak 做过类似的，我们的 UGC 视频上传就是用户上传之后异步转码、再走 CDN 分发，所以这条主线我比较熟。

规模估算

假设我们有 10 亿注册用户，日活 2 亿。先算读，也就是播放。假设每个日活用户一天看 5 个视频，那每天播放请求 = $2 \text{ 亿} \times 5 = 10 \text{ 亿次}$ （也就是日活用户数乘以人均每天看的视频数，等于十亿次播放）。换算成 QPS，播放 QPS = $10 \text{ 亿} / 86400 \approx 11600$ （也就是每天十亿次播放除以一天有 86400 秒，大约每秒一万一千次请求）。考虑到晚高峰，我一般会再乘以 2 到 3 倍算峰值，所以峰值大概三万多 QPS。

再算写，也就是上传。上传的人少得多，假设每天有 100 万个用户各上传 1 个视频，那上传 QPS = $100 \text{ 万} / 86400 \approx 12$ （也就是每天一百万次上传除以 86400 秒，大约每秒 12 次）。你看这里读写比大概是 11600 比 12，差不多一千倍，这就印证了前面说的读多写少，所以我的架构一定是重点优化读路径。

存储也估一下。假设一个视频原始文件平均 300MB，每天 100 万个，那每天新增原始存储 = $100 \text{ 万} \times 300\text{MB} = 300\text{TB/天}$ （也就是每天上传数乘以单个视频大小，等于三百 TB 每天）。但注意转码之后一个视频要存多个清晰度（1080p/720p/480p 等），乘上冗余大概再翻个两三倍，所以实际每天可能要 700TB 到 1PB 的存储增量。一年下来是几百 PB，这个量级只能上对象存储，不可能放数据库或普通文件系统。

API 设计

我会设计得很薄，主要三类。上传这块我不会让客户端把整个视频 POST 到我的业务服务器，那样又慢又占带宽。我会用一个两步的方式：先调 `POST /videos`（客户端带上标题、描述这些 metadata，服务端返回一个 `videoId` 和一个预签名的上传地址 `uploadUrl`），然后客户端拿着这个 `uploadUrl` 直接把视频文件 PUT 到对象存储（也就是 S3 的 `presigned URL`，让客户端绕过业务服务器直传，服务器只管发凭证）。

presigned URL 怎么来的值得说一句，因为它跟直觉不太一样：服务端用自己的云账号密钥做一次 HMAC 签名就生成了，**全程不需要联网调 S3**——因为签名方和验证方共享同一个密钥。签名覆盖了 HTTP 方法、bucket、object key、过期时间，客户端改任何一个签名就失效。所以这个凭证的权限被限制得极窄：只能在有效期内、用指定方法、传到这一个特定的 key。

大文件我会走分片上传（multipart，切成 5-10MB 一片并行传、断了能续传）。这时候流程稍有不同：

`CreateMultipartUpload` 要**服务端**调（它需要云账号凭证，业务校验也在这一步做完），拿到 `uploadId` 之后再为每个分片签一批 URL 发给客户端；`UploadPart` 和 `CompleteMultipartUpload` 都是客户端直连 S3。手机网络不稳定，不支持断点续传体验会很差，所以视频这种大文件基本默认走 multipart；头像封面这类小文件单次 PUT 就够，没必要上这个复杂度。

播放这块是 `GET /videos/{videoId}`，返回 metadata 加上一个 manifest 地址（就是那个描述有哪些清晰度、每段在哪的清单文件）。真正的视频数据不走这个接口，客户端拿到 manifest 后直接去 CDN 拉分片。另外我会有个 `GET /videos/{videoId}/status` 让客户端轮询转码进度，因为上传完不是马上能看的。

数据模型

这里核心的取舍是「什么放哪」。视频文件本身，也就是原始文件和转码后的各种清晰度分片，全部放对象存储（S3 这类），因为它便宜、容量近乎无限、天然适合大文件，而且能直接跟 CDN 对接。绝对不放数据库。

metadata，比如 videoid、上传者 userId、标题、描述、时长、上传时间、转码状态、各清晰度文件在对象存储里的路径，这些放数据库。用哪种库我会看情况：如果只是这些结构化字段、量级也就十亿行，用 MySQL 就够了，按 videoid 做 sharding（也就是按视频 ID 把数据打散到多个库上）。但如果字段很灵活、以后要加各种标签属性，我会考虑用一个 NoSQL 比如 MongoDB。播放量、点赞数这种高频更新又不要求强一致的计数，我会单独用 Redis 扛，异步刷回数据库，不然每次播放都去写主库会把库打爆。

还有一个热点数据的问题，热门视频的 metadata 会被反复读，所以我会数据库前面加一层 Redis cache，videoid 做 key，命中就不打库了。这跟前面说的读多写少是呼应的——读路径上能加缓存的地方我都会加。

高层架构

把数据流串一下。上传路径：客户端先调 `POST /videos` 拿 videoid 和预签名地址——**所有业务校验（鉴权、配额、参数）都在这一步做完**，因为预签名 URL 一旦发出去就收不回来了，只能靠有效期自然过期。同时服务端在这里就把 metadata 记录建好（标题、描述、状态=uploading），后面事件回来时靠 object key 关联。

然后视频文件直传对象存储的「原始桶」。大文件走分片上传，客户端把每片的 ETag 也回报给业务服务器一份——这份不是给对象存储用的（S3 自己有），是为了**断点续传**：客户端崩了重启后问服务端就知道哪些片传过了，跳过它们。

所有分片传完后，客户端调 `CompleteMultipartUpload`（这是对象存储的 API，客户端直连，不过业务服务器），S3 校验 ETag 后把分片合并成一个完整对象，**此刻才发出恰好一次 `ObjectCreated` 事件**，这个事件触发转码流水线。这里有个值得讲的细节：触发点不是“字节传完”而是“客户端主动调 Complete”——**分片在 Complete 之前只是暂存，不构成对象**，所以用户上传到一半放弃、或者传完又反悔，只要不调 Complete，下游就什么都不会发生，过期后由生命周期策略自动清理。转码是重活、耗时又不确定，所以必须异步，绝不能卡在上传请求里，这也是为什么我用 Kafka 把上传和转码解耦——上传服务发完消息就返回，转码 worker 在后台慢慢消费。

转码 worker 从队列里取任务，把原始视频转成多个清晰度、并且切成小分片，同时生成那个 manifest 清单文件，转码结果写回对象存储的「成品桶」，然后更新数据库里这个视频的状态为「就绪」。这里为了扛住峰值，转码 worker 是可以水平扩的一个 worker 池，队列积压了就加机器。

播放路径：客户端调业务服务器拿到 metadata 和 manifest 地址，然后就直接去 CDN 拉视频分片了。CDN 是这里的关键，它把视频缓存到离用户近的边缘节点，第一个用户看的时候从对象存储回源、之后所有人都命中边缘节点，既降延迟又省掉对象存储的出口带宽。播放器根据当前网速在 manifest 里的几个清晰度之间自动切换，这就是码率自适应（也就是 adaptive bitrate，网好放高清、网差自动降码率保证不卡），协议一般用 HLS 或 DASH。

关键取舍与瓶颈

第一个核心难点是转码。它慢、吃 CPU、还可能失败，所以我用队列 + worker 池把它彻底做成异步，上传立刻返回、用户轮询状态。取舍是用户上传完不能马上看，得等转码，但这个换来了系统的稳定性和可扩展性，是值得的。worker 挂了消息还在队列里，重试就行。

第二个是 CDN 和带宽成本。视频系统最大的开销就是出口带宽，如果不用 CDN，所有播放都回源打对象存储，带宽费会爆炸而且延迟高。用了 CDN 之后热门视频基本命中边缘节点，回源率很低。取舍是 CDN 有成本、冷门视频命中率低，但对读多写少、少数视频占大部分流量的场景，这笔钱花得非常值。

第三个是自适应码率带来的存储放大。为了支持多清晰度，一个视频要存好几份，存储直接翻倍甚至更多。取舍就是拿存储成本换播放体验，一般会做，但可以优化——冷门视频只转低清晰度、或者干脆按需转码（有人点了再转），热门视频才全清晰度都备好。

第四点讲扩展，读路径怎么再扛。业务服务器是无状态的，直接水平扩加机器、前面挂 load balancer 就行。数据库这层我靠 Redis cache 挡住绝大部分读，再加读副本分摊。计数类的高频写用 Redis 聚合后批量刷库。真正的大头流量其实根本不碰我的服务器，都被 CDN 消化掉了，所以这个系统扩展的瓶颈往往不在业务逻辑，而在转码算力和 CDN/存储成本。

最后如果面试官还想深入，我会提一句可以聊聊上传的断点续传细节、或者热门视频的预热（提前推到 CDN）、又或者怎么做视频的软删除和存储生命周期管理（冷数据降到便宜的存储层）。这几块 Newsbreak 那边我们也踩过一些坑，可以展开。
